

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSL333 Database Management Systems Lab

LAB MANUAL

PREPARED BY:

Ms. ANITHA L

APPROVED BY:

HOD, CSE

Vision of the Institution

Progress through education

Mission of the Institution

To seek, strive for and scale greater heights of quality education

Vision of the Department

Progress through quality education and become a centre of excellence in computing knowledge

Mission of the Department

To seek, to strive for advancements in technological and social aspects so as to create socially responsible engineers in the field of computer science, who will embark to further reduce gap between science and man-kind.

Program Educational Objectives (PEOs)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO-1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.

PO-2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO-3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO-4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO-5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO-6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO-7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO-8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO-9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.

PO-10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write

effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11:Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12:Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE SYLLABUS**LAB Experiments:**

1. Design a database schema for an application with an ER diagram from a problem description.
2. Creation, modification, configuration, and deletion of databases using UI and SQL Commands.
3. Creation of database schema - DDL (create tables, set constraints, enforce relationships, create indices, delete and modify tables). Export ER diagram from the database and verify relationships (with the ER diagram designed in step 1).
4. Database initialization - Data insert, Data import to a database (bulk import using UI and SQL Commands).
5. Practice SQL commands for DML (insertion, updating, altering, deletion of data, and viewing/querying records based on condition in databases).
6. Implementation of built-in functions in RDBMS.
7. Implementation of various aggregate functions in SQL.
8. Implementation of Order By, Group By & Having clause .
9. Implementation of set operators nested queries, and join queries .
10. Implementation of queries using temp tables.
11. Practice of SQL TCL commands like Rollback, Commit, Savepoint .
12. Practice of SQL DCL commands for granting and revoking user privileges .
13. Practice of SQL commands for creation of views and assertions.
14. Implementation of various control structures like IF-THEN, IF-THEN-ELSE, IF-THEN-ELSIF, CASE, WHILE using PL/SQL.
15. Creation of Procedures, Triggers and Functions.
16. Creation of Packages.
17. Creation of Cursors.
18. Creation of PL/SQL blocks for exception handling.
19. Database backup and restore using commands.
20. Query analysis using Query Plan/Show Plan.
21. Familiarization of NoSQL Databases and CRUD operations.
22. Design a database application using any front end tool for any problem selected. The application constructed should have five or more tables.

Program: B. Tech	Department: Computer Science and Engineering
Course: Database Management Systems Lab	Semester: S5 Batch: C Credits: 2
Course Code: CSL333	Regulation: 2019
Course Area/Domain: General Engineering	L: 0 T: 0 TW/SL: 1 P: 3
Corresponding Lab Course Code (If Any): Nil	Lab Course Name: Nil

COURSE PRE-REQUISITES:

CODE	COURSE NAME	DESCRIPTION	SEM
CST204	Database Management Systems	A sound knowledge of the basics of relational DBMS	S4

COURSE OUTCOMES:

SL NO	DESCRIPTION
CO1	Design database schema for a given real world problem-domain using standard design and modeling approaches.
CO2	Construct queries using SQL for database creation, interaction, modification, and updation.
CO3	Design and implement triggers, cursors, procedures, functions, and control structures using PL/SQL.
CO4	Perform CRUD operations in NoSQL Databases.
CO5	Develop database applications using front-end tools and back-end DBMS.

CO - PO/PSO MAPPING:

CO Number	PO STRENGTH												PSO STRENGTH		
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO1	PSO2	
CO1	2	2	3		2			2		2		2	2	2	
CO2	2	2	2		3			2		2		2	2	2	
CO3	2	2	2	2	3			2		2		2	2	2	
CO4	2	2	2	2	3			2		2		2	2	2	
CO5	3	3	3	3	3	2		2	2	2	2	2	3	3	
Total															

GENERAL INSTRUCTIONS (for the lab)**Preparation:**

- Software Installation: Ensure you have the required DBMS software installed (e.g., MySQL, PostgreSQL, PLSQL, Oracle, or SQL Server). Download and install the appropriate community or student edition.
- Client Tools: Install a suitable client or IDE for interacting with the database server (e.g., MySQL Workbench, pgAdmin, SQL Developer, or the DBMS vendor's command-line interface).
- Familiarization: Review the basic SQL syntax for Data Definition Language (DDL) and Data Manipulation Language (DML).

Lab Setup:

- Directory Creation: Create a new directory for the DBMS lab session (e.g., DBMS_Lab_S1_CSE_A) to store all your .sql script files and reports.
- Database Access: Ensure you have the necessary credentials (username and password) and connection details (hostname, port) to access the database server.
- Server Check: Ensure an Oracle Database instance (like Oracle XE) is running on your Windows machine.
- IDE: Use Oracle SQL Developer or a similar IDE. Ensure you have a working connection to your Oracle schema.
- Debugging Tool: Within your SQL Developer worksheet, always run SET SERVEROUTPUT ON; to enable output from DBMS_OUTPUT.PUT_LINE, which is crucial for testing PL/SQL logic.

Documentation:

- Script Header: Start each SQL script file with comments (--) that include your name, date, the Experiment/Program Number, and a brief description of the script's purpose.
- Inline Comments: Use comments to explain the purpose of complex queries, stored procedures, triggers, or any part of the database design (like schema creation or integrity constraints).

Coding Standards:

- SQL Standards: While there is no official PEP 8 equivalent, follow consistent SQL coding standards:

- Use uppercase for SQL keywords (e.g., SELECT, FROM, WHERE).
- Use meaningful and descriptive names for tables, columns, and other database objects.
- Use a consistent casing style (e.g., snake_case or PascalCase) for identifiers.
- Indentation and Formatting: Properly indent your SQL queries, especially for multi-line commands like CREATE TABLE, JOIN queries, or PL/SQL blocks, to improve readability.

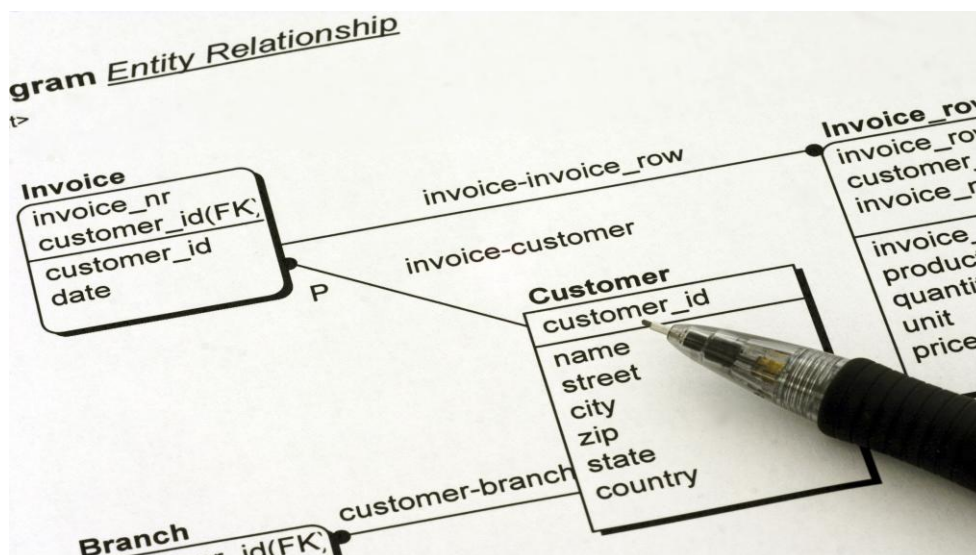
Testing and Debugging:

- Verify Schema: After using Data Definition Language (DDL) commands (CREATE, ALTER, DROP), always use commands like DESCRIBE <table_name> or its equivalent to verify the structure.
- Test Data Integrity: After using Data Manipulation Language (DML) commands (INSERT, UPDATE, DELETE), use a SELECT query to immediately verify the data has been modified as expected.
- Transaction Control: Practice using Transaction Control Language (TCL) commands like COMMIT and ROLLBACK to test operations safely before permanently applying changes.

BEST PRACTICES

Understand the Basics:

- Conceptual Clarity: Ensure a solid understanding of fundamental concepts like Database Architecture, E-R Modeling



Relational Algebra, and Normalization (1NF, 2NF, 3NF).

- SQL Command Types: Differentiate clearly between DDL (Data Definition), DML (Data Manipulation), DCL (Data Control), and TCL (Transaction Control) commands.

Plan Your Database Design:

- E-R Diagram First: Before writing any CREATE TABLE statements, design the database structure using an Entity-Relationship (E-R) diagram.
- Normalization: Apply appropriate normalization techniques to eliminate data redundancy and anomalies before implementing the schema.

Write Readable SQL:

- Integrity Constraints: Always define Primary Keys, Foreign Keys, NOT NULL, UNIQUE, and CHECK constraints when creating tables to enforce data integrity at the database level.
- Efficient Queries: Learn to write optimized SQL queries. Use JOIN operations (Inner, Outer, Self) correctly instead of multiple simple queries where a join is more efficient.
- Use Variables/PL-SQL: For complex logic, practice using Stored Procedures, Functions, and Triggers in the procedural language (like PL/SQL or T-SQL) provided by your RDBMS.

Data Management:

- Backup and Restore: Understand and practice basic database backup and restore operations, even if it's just for your lab environment.
- Security (DCL): Understand the use of Data Control Language (DCL) commands like GRANT and REVOKE to manage user permissions and access control.

Practice and Resources:

- Manual/Problem Sets: Always refer to the specific lab manual and the problem statement before starting an experiment.
- Online Documentation: Utilize the official documentation for your RDBMS (e.g., Oracle or MySQL documentation) as your primary reference for syntax and features.

INDEX

Exp No.	Expt. Name	Page Number
1	University Management System Er Diagram	18
2	Creation, Modification, Configuration, And Deletion Of Databases Using Ui And Sql Commands	21
3	Creation Of Database Schema Using Ddl And Er Diagram Verification	25
4	Data Insertion And Bulk Import Using Dml Commands And Ui Tools	29
5	Sql Dml: Essential Data Manipulation Operations	34
6	Implementation Of Built-In Functions In Rdbms	40
7	Implementation Of Various Aggregate Functions In Sql	43
8	Implementation Of Order By, Group By And Having Clause In Sql	47
9	Implementation Of Set Operators, Nested Queries, And Join Queries	50
10	Implementation Of Queries Using Temp Tables	57
11	Implementation Of Sql Tcl Commands	61
12	Implementation Of Sql Dcl Commands	66
13	Creation And Manipulation Of Tables, Views, And Assertions In Sql	71
14	Implementation Of Control Structures In Pl/Sql	77
15	Creation Of Packages	81
16	Creation Of Procedures, Triggers And Functions	85
17	Creation Of Cursors	88
18	Creation Of Pl/Sql Blocks For Exception Handling	91
19	Database Backup And Restore Using Commands	93
20	Query Analysis Using Query Plan / Show Plan	95
21	Familiarization Of Nosql Databases And Crud Operations	98
22	Database Application Using Front End Tool	101

INTRODUCTION TO SQL

History of SQL

Dr. E. F. Codd published the paper, "A Relational Model of Data for Large Shared Data Banks", in June 1970 in the Association of Computer Machinery (ACM) journal, Communications of the ACM. Codd's model is now accepted as the definitive model for relational database management systems (RDBMS). The language, Structured English Query Language ("SEQUEL") was developed by IBM Corporation, Inc., to use Codd's model. SEQUEL later became SQL (still pronounced "sequel"). In 1979, Relational Software, Inc. (now Oracle Corporation) introduced the first commercially available implementation of SQL. Today, SQL is accepted as the standard RDBMS language.

How SQL Works

The strengths of SQL provide benefits for all types of users, including application programmers, database administrators, managers, and end users. Technically speaking, SQL is a data sub-language. The purpose of SQL is to provide an interface to a relational database such as Oracle, and all SQL statements are instructions to the database. SQL differs from general-purpose programming languages like C and BASIC. Among the features of SQL are the following:

1. It processes sets of data as groups rather than as individual units.
2. It provides automatic navigation to the data.
3. It uses statements that are complex and powerful individually, and that therefore stand alone.

Flow-control statements were not part of SQL originally, but they are found in the recently accepted optional part of SQL, ISO/IEC 9075-5: 1996. Flow-control statements are commonly known as "persistent stored modules" (PSM), and Oracle's PL/SQL extension to SQL is similar to PSM.

Essentially, SQL lets you work with data at the logical level. You need to be concerned with the implementation details only when you want to manipulate the data. For example, to retrieve a set of rows from a table, you define a condition used to filter the rows. All rows satisfying the condition are retrieved in a single step and can be passed as a unit to the user, to another SQL statement, or to an application. You need not deal with the rows one by one, nor do you have to worry about how they are physically stored or retrieved.

All SQL statements use the optimizer, a part of Oracle that determines the most efficient means of accessing the specified data. Oracle also provides techniques you can use to make the optimizer perform its job better.

SQL provides statements for a variety of tasks, including:

1. Querying data
2. Inserting, updating, and deleting rows in a table
3. Creating, replacing, altering, and dropping objects
4. Controlling access to the database and its objects
5. Guaranteeing database consistency and integrity

SQL unifies all of the above tasks in one consistent language.

Common Language for All Relational Databases

All major relational database management systems support SQL, so you can transfer all skills you have gained with SQL from one database to another. In addition, all programs written in SQL are portable. They can often be moved from one database to another with very little modification.

Summary of SQL Statements

SQL statements are divided into these categories:

1. Data Definition Language (DDL) Statements
2. Data Manipulation Language (DML) Statements
3. Transaction Control Statements (TCL)
4. Session Control Statement
5. System Control Statement

Managing Tables

A table is a data structure that holds data in a relational database. A table is composed of rows and columns. A table can represent a single entity that you want to track within your system. This type of a table could represent a list of the employees within your organization, or the orders placed for your company's products.

A table can also represent a relationship between two entities. This type of a table could portray the association between employees and their job skills, or the relationship of products to orders. Within the tables, foreign keys are used to represent relationships.

Creating Tables

To create a table, use the SQL command `CREATETABLE`.

Syntax:

CREATE TABLE <TABLE NAME>(<FIELD NAME ><DATA TYPE><[SIZE]>,<.....>)

Altering Tables

Alter a table in an Oracle database for any of the following reasons:

1. To add one or more new columns to the table
2. To add one or more integrity constraints to a table
3. To modify an existing column's definition (datatype, length, default value, and NOTNULL integrity constraint)
4. To modify data block space usage parameters (PCTFREE, PCTUSED)
5. To modify transaction entry settings (INITRANS, MAXTRANS)
6. To modify storage parameters (NEXT, PCTINCREASE, etc.)
7. To enable or disable integrity constraints associated with the table
8. To drop integrity constraints associated with the table

When altering the column definitions of a table, you can only increase the length of an existing column, unless the table has no records. You can also decrease the length of a column in an empty table. For columns of data type CHAR, increasing the length of a column might be a time consuming operation that requires substantial additional storage, especially if the table contains many rows. This is because the CHAR value in each row must be blank-padded to satisfy the new column length.

If you change the datatype (for example, from VARCHAR2 to CHAR), then the data in the column does not change. However, the length of new CHAR columns might change, due to blank-padding requirements.

Altering a table has the following implications:

- If a new column is added to a table, then the column is initially null. You can add a column with a NOT NULL constraint to a table only if the table does not contain any rows.
- If a view or PL/SQL program unit depends on a base table, then the alteration of the base table might affect the dependent object, and always invalidates the dependent object.

Privileges Required to Alter a Table

To alter a table, the table must be contained in your schema, or you must have either the ALTER object privilege for the table or the ALTER ANY TABLE system privilege.

Dropping Tables

Use the SQL command DROP TABLE to drop a table. For example, the following statement drops the

EMP_TAB table:

If the table that you are dropping contains any primary or unique keys referenced by foreign keys to other tables, and if you intend to drop the FOREIGN KEY constraints of the child tables, then include the CASCADE option in the DROP TABLE command.

Oracle Built-In Datatypes

A datatype associates a fixed set of properties with the values that can be used in a column of a table or in an argument of a procedure or function. These properties cause Oracle to treat values of one datatype differently from values of another datatype. For example, Oracle can add values of sNUMBER datatype, but not values of RAW datatype.

Oracle supplies the following built-in data types: character data types

1. CHAR
2. NCHAR
3. VARCHAR2 and VARC
4. NVARCHAR2
5. CLOB
6. NCLOB
7. LONG
 - a. NUMBER datatype
 - b. DATE datatype
 - c. Binary datatypes
8. BLOB
9. BFILE
10. RAW
11. LONG RAW

Another datatype, ROWID, is used for values in the ROWID pseudocolumn, which represents the unique address of each row in a table.

Table summarizes the information about each Oracle built-in datatype.

Summary of Oracle Built-In Data types

Using Character Data types

Use the character data types to store alphanumeric data.

1. CHAR and NCHAR data types store fixed-length character strings.

2. VARCHAR2 and NVARCHAR2 data types store variable-length character strings. (The VARCHAR datatype is synonymous with the VARCHAR2 datatype.)
3. CLOB and NCLOB data types store single-byte and multi-byte character strings of up to four gigabytes.
4. The LONG datatype stores variable-length character strings containing up to two gigabytes, but with many restrictions.
5. This data type is provided for backward compatibility with existing applications; in general, new applications should use CLOB and NCLOB data types to store large amounts of character data.

When deciding which datatype to use for a column that will store alphanumeric data in a table, consider the following points of distinction:

Space Usage

1. To store data more efficiently, use the VARCHAR2 datatype. The CHAR datatype blank-pads and stores trailing blanks up to a fixed column length for all column values, while the VARCHAR2 datatype does not blank-pad or store trailing blanks for column values.
2. Use the CHAR data type when you require ANSI compatibility in comparison semantics (when trailing blanks are not important in string comparisons). Use the VARCHAR2 when trailing blanks are important in string comparisons.

Comparison Semantics

Use the CHAR data type when you require ANSI compatibility in comparison semantics (when trailing blanks are not important in string comparisons). Use the VARCHAR2 when trailing blanks are important in string comparisons.

Future Compatibility

- The CHAR and VARCHAR2 data types are and will always be fully supported. At this time, the VARCHAR datatype automatically corresponds to the VARCHAR2 datatype and is reserved for future use.
- CHAR, VARCHAR2, and LONG data is automatically converted from the database character set to the character set defined for the user session by the NLS_LANGUAGE parameter, where these are different.

Using the NUMBER Datatype

Use the NUMBER datatype to store real numbers in a fixed-point or floating-point format. Numbers using this data type are guaranteed to be portable among different Oracle platforms, and offer up to

38 decimal digits of precision. You can store positive and negative numbers of magnitude 1×10^{-130} to 9.99×10^{125} , as well as zero, in a NUMBER column.

For numeric columns you can specify the column as a floating-point number:

Column_name NUMBER

Or, you can specify a precision (total number of digits) and scale (number of digits to the right of the decimal point):

Column_name NUMBER (<precision>, <scale>)

Although not required, specifying the precision and scale for numeric fields provides extra integrity checking on input. If a precision is not specified, then the column stores values as given. The table shows examples of how data would be stored using different scale factors.

Using the DATE Datatype

Use the DATE datatype to store point-in-time values (dates and times) in a table. The DATE datatype stores the century, year, month, day, hours, minutes, and seconds.

Oracle uses its own internal format to store dates. Date data is stored in fixed-length fields of seven bytes each, corresponding to century, year, month, day, hour, minute, and second.

Date Format

For input and output of dates, the standard Oracle default date format is DD-MON-YY.

For example: '13-NOV-92'

To change this default date format on an instance-wide basis, use the NLS_DATE_FORMAT parameter. To change the format during a session, use the ALTER SESSION statement. To enter dates that are not in the current default date format, use the TO_DATE function with a format mask.

For example:

TO_DATE ('November 13, 1992', 'MONTH DD, YYYY')

If the date format DD-MON-YY is used, then YY indicates the year in the 20th century (for example, 31-DEC-92 is December 31, 1992). If you want to indicate years in any century other than the 20th century, then use a different format mask, as shown above.

Time Format

Time is stored in 24-hour format #HH:MM:SS. By default, the time in a date field is 12:00:00 A.M. (midnight) if no time portion is entered. In a time-only entry, the date portion defaults to the first day of the current month. To enter the time portion of a date, use the TO_DATE function with a format mask indicating the time portion, as in:

```
INSERT INTO Birthdays_tab (bname, bday) VALUES ('ANNIE',TO_DATE('13-NOV-92 10:56 A.M.','DD-MON-YY HH:MI A.M.'));
```

To compare dates that have time data, use the SQL function TRUNC if you want to ignore the time component.

Use the SQL function SYSDATE to return the system date and time. The FIXED_DATE initialization parameter allows you to set SYSDATE to a constant; this can be useful for testing.

INTRODUCTION TO ER MODEL

The ER model represents real world situations using concepts, which are commonly used by people. It allows defining a representation of the real world at a logical level. ER model has no facilities to describe machine-related aspects.

In the ER model the logical structure of data is captured by indicating the grouping of data into entities. The ER model also supports a top-down approach by which details can be given in successive stages.

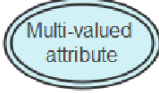
Entity: An entity is something which is described in the database by storing its data, it may be a concrete entity or a conceptual entity.

Entity set: An entity set is a collection of similar entities.

Attribute: An attribute describes a property associated with entities. Attributes will have a name and a value for each entity.

Domain: A domain defines a set of permitted values for an attribute

SYMBOLS IN E-R DIAGRAM

	Entity		Attribute
	Weak Entity		Key Attribute
	Associative Entity		Key Attribute
	Relationship		Key Attribute
	Identifying Relationship		Derived Attribute
	Mandatory Relationship		Optional Relationship
	Partial Participation		Total Participation

EXP NO:1**UNIVERSITY MANAGEMENT SYSTEM ER DIAGRAM****AIM**

A university registrar's office maintains data about the following entities: (a) courses, including number, title, credits, syllabus, and prerequisites; (b) course offerings, including course number, year, semester, section number, instructor(s), timings, and classroom; (c) students, including student-id, name, and program; and (d) instructors, including identification number, name, department, and title. Further, the enrollment of students in courses and grades awarded to students in each course they are enrolled for must be appropriately modeled. Construct an E-R diagram for the registrar's office. Document all assumptions that you make about the mapping constraints.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Analyze a problem description and extract necessary entities and attributes.
2. Identify appropriate relationships and cardinalities among entities.
3. Construct a clear, normalized ER diagram based on requirements.
4. Develop a database schema from the ER diagram.
5. Recognize the importance of keys (primary, foreign) in relational design.
6. Apply conceptual modeling techniques to design real-world databases.
7. Document assumptions when real-world data is incomplete or ambiguous.
8. Translate conceptual models into relational models ready for implementation.

DESCRIPTION

Designing a database schema from a problem description is a fundamental step in database development. The process begins with understanding the real-world scenario, identifying important objects(entities),determining

their properties (attributes), and analyzing how these entities interact (relationships).To represent this visually, an Entity–Relationship (ER) diagram is created.

An ER diagram typically includes:

- Entities: Objects like Student, Course, Instructor, etc.

- Attributes: Properties like student_id, course_title, instructor_name.
- Primary keys: Unique identifiers for each entity.
- Relationships: How entities are connected (e.g., Student *enrolls* in Course).
- Cardinality: 1:1, 1:N, or M:N constraints between entities.

After designing the ER diagram, it is mapped to a logical relational schema, which is later implemented in a database system (SQL).

This process ensures:

- Data consistency
- Avoidance of redundancy
- Clear structure for storage and retrieval
- Correct modeling of real-world requirements

OUTPUT:

The main entity sets are student, course, course-offering, and instructor. The entity set course-offering is a weak entity set dependent on course. The assumptions made are :

- A class meets only at one particular place and time. This E-R diagram cannot model a class meeting at different places at different times.
- There is no guarantee that the database does not have two classes meeting at the same place and time

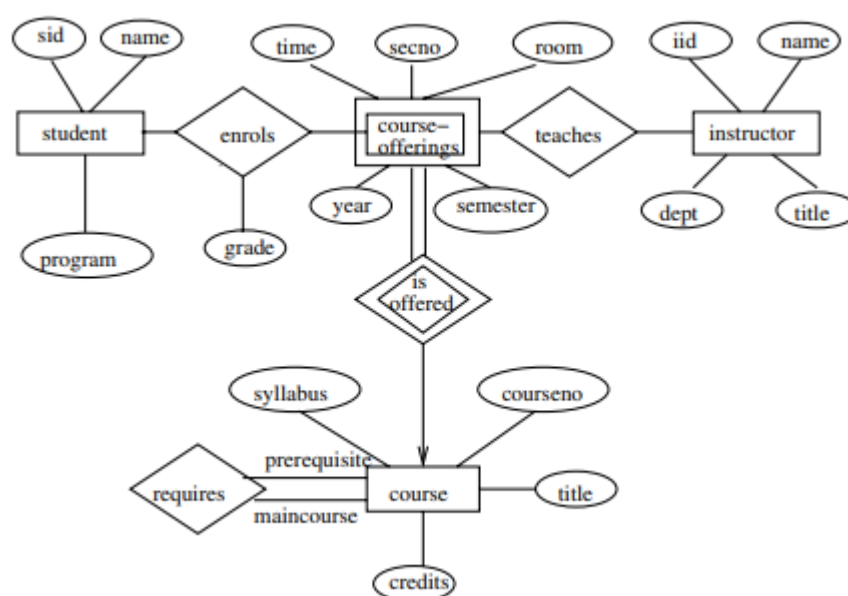


Fig. ER Diagram for a University

MODEL VIVA QUESTIONS

1. What is an ER diagram?
2. Define an entity in the context of database design.
3. What is an attribute? Give examples from your design
4. What is the purpose of a primary key?
5. What is a foreign key and why is it used?
6. What is meant by cardinality in an ER diagram?
7. Explain the difference between 1:1, 1:N, and M:N relationships.

RESULT:

ER diagram has been drawn successfully. By constructing an ER diagram, I was able to apply a standard design and modelling approach.

EXP NO:2**CREATION, MODIFICATION, CONFIGURATION, AND DELETION OF
DATABASES USING UI AND SQL COMMANDS****AIM**

To study and perform the creation, modification, configuration, and deletion of databases using both graphical user interface (UI) tools and SQL commands.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

1. Create a database using SQL commands and UI tools.
2. Modify database properties using SQL commands.
3. Configure database settings as required.
4. Delete databases safely using SQL commands.
5. Understand the role of UI-based database management tools.

DESCRIPTION

A database is an organized collection of structured data stored electronically in a computer system. Database Management Systems (DBMS) provide two primary methods for managing databases:

- Graphical User Interface (UI) Tools: Tools such as MySQL Workbench, SQL Server Management Studio (SSMS), or pgAdmin allow users to perform database operations through menus, wizards, and forms without writing SQL code directly.
- SQL Commands: The Structured Query Language (SQL) provides Data Definition Language (DDL) commands such as CREATE, ALTER, and DROP to manage the structure of databases directly through code.

Both approaches ultimately perform the same underlying operations at the database engine level, and understanding both equips a database administrator to work efficiently in different environments.

PROCEDURE

Step-by-Step Procedure in MySQL Workbench

1. Open MySQL Workbench

- Launch MySQL Workbench
- Click on the required MySQL connection
- Enter the password if prompted

2. Create a Database

1. In the Navigator panel, right-click on Schemas
2. Select Create Schema
3. Enter a schema (database) name
4. Click Apply
5. Review the SQL statement
6. Click Apply → Finish

3. Modify a Database

- Right-click on the created database
- Select Schema Inspector or Alter Schema
- Modify properties such as:
 - Default character set
 - Collation
- Click Apply to save changes

4. Configure Database (Add Tables / Settings)

6. Expand the database
7. Right-click Tables
8. Select Create Table
9. Define:
 - Table name
 - Columns
 - Data types
 - Primary key
10. Click Apply

5. Delete a Database

1. Right-click on the database
2. Select Drop Schema
3. Check Drop the schema
4. Click Apply

5. Confirm by clicking Apply → Finish

SQL QUERY QUESTIONS

1. How do you create a new database using SQL?
2. How can you view the list of available databases?
3. How do you modify database properties?
4. How can a database be deleted using SQL commands?

SQL QUERY

1. How do you create a new database using SQL?

CREATE DATABASE student_db;

```
mysql> CREATE DATABASE student_db;  
Query OK, 1 row affected (0.01 sec)
```

2. How can you view the list of available databases?

SHOW DATABASES;

```
mysql> SHOW DATABASES;  
+-----+  
| Database          |  
+-----+  
| information_schema |  
| mysql              |  
| performance_schema |  
| student_db         |  
| sys                |  
+-----+  
5 rows in set (0.00 sec)
```

3. How do you modify database properties?

USE student_db;

ALTER DATABASE student_db CHARACTER SET utf8mb4;

```
mysql> USE student_db;  
Database changed
```

```
mysql> ALTER DATABASE student_db CHARACTER SET utf8mb4;  
Query OK, 1 row affected (0.00 sec)
```

4. How can a database be deleted using SQL commands?

```
DROP DATABASE student_db;
```

```
mysql> DROP DATABASE student_db;  
Query OK, 0 rows affected (0.02 sec)
```

MODEL VIVA QUESTIONS

1. What is a database?
2. What command is used to create a database?
3. How do you display all databases in MySQL?
4. What is the purpose of the ALTER DATABASE command?
5. What happens when a database is dropped?

RESULT:

Thus, the creation, modification, configuration, and deletion of databases were successfully performed using both UI tools and SQL commands

EXP NO:3**CREATIONF DATABASE SCHEMA USING DDL AND ER DIAGRAM**
VERIFICATION**AIM**

To create a database schema using Data Definition Language (DDL) commands by creating tables, defining constraints, enforcing relationships, creating indices, and modifying/deleting tables. Also, to export the ER diagram from the database and verify the relationships with the ER diagram designed earlier.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Understand DDL commands in SQL
- Create and manage database schemas
- Define primary key, foreign key, and other constraints
- Enforce relationships between tables
- Create and use indexes
- Modify and delete tables
- Generate and verify ER diagrams from the database

DESCRIPTION

A database schema is the logical structure that defines how data is organized within a database, including tables, columns, data types, constraints, and the relationships between different tables. Data Definition Language (DDL) is a subset of SQL used to define, create, and modify this structure. Common DDL commands include CREATE TABLE, ALTER TABLE, and DROP TABLE.

While creating tables, various constraints are applied to maintain data accuracy and integrity:

- PRIMARY KEY – uniquely identifies each record in a table and does not allow NULL or duplicate values.
- FOREIGN KEY – establishes a link between two tables by referencing the primary key of another table, thereby enforcing referential integrity and relationships such as one-to-one, one-to-many, or many-to-many.
- UNIQUE – ensures that all values in a column are distinct.

- NOT NULL – ensures that a column cannot have a NULL value.
- CHECK – restricts the range or set of values that can be entered into a column.
- DEFAULT – assigns a default value to a column when no value is specified.

Indices are special lookup structures created on one or more columns of a table to speed up data retrieval operations. While indices improve the performance of SELECT queries, they may slightly slow down INSERT, UPDATE, and DELETE operations due to the need to update the index as well.

An Entity-Relationship (ER) Diagram is a graphical representation of entities (tables), their attributes (columns), and the relationships between them, typically designed during the database design phase before implementation. Once the schema is implemented using DDL commands, most modern DBMS tools (such as MySQL Workbench or SSMS) provide a "Reverse Engineer" or "Export ER Diagram" feature that automatically generates an ER diagram from the existing database structure. This generated diagram can then be compared with the originally designed ER diagram to verify that all entities, attributes, primary keys, foreign keys, and relationships have been correctly and consistently implemented in the actual database.

SQL QUERY QUESTIONS

1. Create a database schema for a student management system.
2. Create tables with appropriate constraints.
3. Enforce relationships using primary and foreign keys.
4. Create an index on a table column.
5. Modify a table structure.
6. Delete a table from the database.
7. Export and verify the ER diagram with the designed ER model.

SQL QUERY

1. Create a database schema for a student management system.

```
CREATE DATABASE college_db;  
USE college_db;
```
2. Create tables with appropriate constraints.

```
CREATE TABLE department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL UNIQUE  
);
```
3. Enforce relationships using primary and foreign keys.

```
CREATE TABLE student (  

```

```
student_id INT PRIMARY KEY,  
student_name VARCHAR(50) NOT NULL,  
email VARCHAR(100) UNIQUE,  
dept_id INT,  
CONSTRAINT fk_dept  
FOREIGN KEY (dept_id) REFERENCES department(dept_id)  
);
```

4. Create an index on a table column.

```
CREATE INDEX idx_student_name  
ON student(student_name);
```

5. Modify a table structure.

```
ALTER TABLE student  
ADD phone_number VARCHAR(15);
```

6. Delete a table from the database.

```
DROP TABLE student;
```

```
mysql> CREATE DATABASE college_db;  
Query OK, 1 row affected (0.01 sec)  
  
mysql> USE college_db;  
Database changed  
  
mysql> CREATE TABLE department (  
-> dept_id INT PRIMARY KEY,  
-> dept_name VARCHAR(50) NOT NULL UNIQUE  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE student (  
-> student_id INT PRIMARY KEY,  
-> student_name VARCHAR(50) NOT NULL,  
-> email VARCHAR(100) UNIQUE,  
-> dept_id INT,  
-> CONSTRAINT fk_dept  
-> FOREIGN KEY (dept_id) REFERENCES department(dept_id)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE INDEX idx_student_name
      -> ON student(student_name);
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE student
      -> ADD phone_number VARCHAR(15);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> DROP TABLE student;
Query OK, 0 rows affected (0.02 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_college_db |
+-----+
| department            |
+-----+
1 row in set (0.00 sec)
```

MODEL VIVA QUESTIONS

1. What is DDL?
2. Which DDL command is used to create a database?
3. What is a primary key?
4. What is a foreign key?
5. Why are constraints used in tables?

RESULT

The database schema was successfully created using DDL commands, all constraints and relationships were enforced correctly, and the exported ER diagram was verified to match the designed ER model.

EXP NO:4**DATA INSERTION AND BULK IMPORT USING DML COMMANDS AND UI TOOLS****AIM:**

To understand how to insert data into database tables, perform bulk data import using UI and SQL commands, and verify successful data initialization using SQL queries.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Perform data insertion using SQL INSERT commands.
- Import large datasets using bulk import tools or LOAD DATA SQL command.
- Understand the purpose of data initialization in database implementation.
- Validate inserted data using SELECT queries.
- Solve real-time problems related to data import and integrity.

DESCRIPTION

Database initialization refers to loading initial data into tables after creating the database schema.

This includes:

1. Data Insert – adding individual rows using SQL INSERT commands.
2. Bulk Import – uploading large datasets quickly using DBMS UI tools (like MySQL Workbench, pgAdmin) or SQL commands such as LOAD DATA INFILE or COPY.
3. Verification – checking whether data is correctly inserted using SELECT queries.

This process is essential before starting operations like searching, updating, or analyzing data.

SQL QUERY QUESTIONS

1. Insert a new course into the Courses table.
2. Insert multiple students into the Students table.
3. Insert enrollment records by referencing foreign keys.
4. Update a student's email ID.
5. Delete a specific course.

6. Display all students enrolled in a specific course.
7. Count the total number of students.
8. Show all courses with more than 3 credits.
9. Perform bulk import of data into the Students table.
10. Verify imported data using a SELECT query.

SQL QUERY

To insert data to tables used in experiment no 2 using insert commands.

1. Insert a new course into the Courses table.

```
INSERT INTO Courses (course_id, course_name, credits, description)
VALUES
(101, 'Database Systems', 4, 'Relational DBMS concept'),
(102, 'Computer Networks', 3, 'Network fundamentals'),
(103, 'Python Programming', 4, 'Programming basics and OOP');
```

```
SELECT * FROM Courses;
```

course_id	course_name	credits	description
101	Database Systems	4	Relational DBMS concept
102	Computer Networks	3	Network fundamentals
103	Python Programming	4	Programming basics

2. Insert multiple students into the Students table.

```
INSERT INTO Students (student_id, full_name, email, dob)
VALUES
(1, 'Rahul Kumar', 'rahul@gmail.com', '2002-05-10'),
(2, 'Aiswarya KP', 'aiswarya@gmail.com', '2001-12-20'),
(3, 'John Mathew', 'john@gmail.com', '2000-08-15');
```

```
SELECT * FROM Student;
```

student_id	full_name	email	dob
1	Rahul Kumar	rahul@gmail.com ↗	2002-05-10
2	Aiswarya KP	aiswarya@gmail.com ↗	2001-12-20
3	John Mathew	john@gmail.com ↗	2000-08-15

3. Insert enrollment records by referencing foreign keys.

```
INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date)
VALUES
(501, 1, 101, '2024-01-10'),
(502, 2, 101, '2024-01-12'),
(503, 3, 103, '2024-01-15');
```

```
SELECT * FROM Enrollments;
```

enrollment_id	student_id	course_id	enrollment_date
501	1	101	2024-01-10
502	2	101	2024-01-12
503	3	103	2024-01-15

4. Update a student's email ID.

```
UPDATE Students
SET email = 'rahul_new@gmail.com'
WHERE student_id = 1;
SELECT * FROM Students WHERE student_id = 1;
```

student_id	full_name	email	dob
1	Rahul Kumar	rahul_new@gmail.com ↗	2002-05-10

5. Delete a specific course.

```
DELETE FROM Courses
WHERE course_id = 102;

SELECT * FROM Courses;
```

course_id	course_name	credits	description
101	DBMS	4	Intro to DBMS
103	Computer Networks	3	Networking fundamentals

6. Display all students enrolled in a specific course.

```
SELECT s.student_id, s.full_name, s.email, e.course_id
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

student_id	full_name	email	course_id
1	Rahul Kumar	rahul_new@gmail.com ↗	101
2	Aiswarya KP	aiswarya@gmail.com ↗	101

7. Count the total number of students.

```
SELECT COUNT(*) FROM Students;
```

COUNT(*)
103

8. Show all courses with more than 3 credits.

```
SELECT * FROM Courses
WHERE credits > 3;
```

course_id	course_name	credits	description
101	DBMS	4	Intro to DBMS
104	Data Science	5	Basics of Data Sci

9. Perform bulk import of data into the Students table.

Create a file named students.csv

Add the data in the following format (with header row):

```
student_id,full_name,email,dob
3,Anu Nair,anu@gmail.com,2001-08-14
```

4,Manu R,manu@gmail.com,2002-01-22

5,Riya Das,riya@gmail.com,2003-11-05

SQL Bulk Import Command:

```
LOAD DATA INFILE '/path/students.csv'
INTO TABLE Students
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

10. Verify imported data using a SELECT query.

```
SELECT * FROM Students;
```

student_id	full_name	email	dob
1	Rahul Kumar	rahul_new@gmail.com ↗	2002-05-10
2	Aiswarya KP	aiswarya@gmail.com ↗	2003-06-18
3	Anu Nair	anu@gmail.com ↗	2001-08-14
4	Manu R	manu@gmail.com ↗	2002-01-22
5	Riya Das	riya@gmail.com ↗	2003-11-05

MODEL VIVA QUESTIONS

1. What is the purpose of data initialization?
2. What is bulk import?
3. Why do we use INSERT INTO command?
4. What happens if a foreign key value doesn't exist in the parent table?
5. What is the difference between INSERT and BULK IMPORT?

RESULT:

The database was successfully initialized by inserting individual records, performing bulk data import, and verifying the inserted data. Students learned how to use SQL commands for data insertion, manage large dataset imports, and validate data integrity using SQL queries.

EXP NO:5**SQL DML: ESSENTIAL DATA MANIPULATION OPERATIONS****AIM:**

To practice and execute SQL DML (Data Manipulation Language) commands such as insertion, updating, altering, deletion of data, and querying records based on specific conditions in a database.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Understand and apply INSERT, UPDATE, DELETE, and SELECT queries.
- Modify existing data correctly and safely inside database tables.
- Apply conditions using WHERE, LIKE, BETWEEN, and comparison operators.
- Retrieve filtered and meaningful information from a database.
- Understand the role of DML commands in manipulating real-time data.
- Distinguish between DDL and DML operations.

DESCRIPTION

Data Manipulation Language (DML) is a subset of SQL used to manage and manipulate the data stored within database tables, as opposed to Data Definition Language (DDL), which deals with the structure of the database. DML commands allow users to insert, retrieve, update, and delete data, making them essential for day-to-day interaction with a database.

The primary DML commands are:

- INSERT – used to add new records (rows) into a table. The INSERT INTO statement specifies the table name, the columns to be filled, and the corresponding values.
- SELECT – used to retrieve data from one or more tables. It can be combined with clauses such as:
 - WHERE – to filter records based on specific conditions.
 - ORDER BY – to sort the result set in ascending or descending order.
 - DISTINCT – to eliminate duplicate records.
 - Logical and comparison operators (AND, OR, NOT, =, <, >, BETWEEN, IN, LIKE) – to build more precise and complex conditions.

- **UPDATE** – used to modify existing records in a table. The SET clause specifies the new values, and the WHERE clause ensures that only the intended records are updated; omitting WHERE updates all rows in the table.
- **DELETE** – used to remove one or more records from a table based on a specified condition using the WHERE clause. Without a WHERE clause, all records in the table are deleted.
- **ALTER** (technically a DDL command but often practiced alongside DML) – used to modify the structure of an existing table, such as adding, modifying, or dropping columns, without affecting the existing data unnecessarily.

SQL QUERY QUESTIONS

Consider the employee database created in Find results for the following questions

1. Find the names of all employees who work for SBI.
2. Find all employees in the database who live in the same cities as the companies for which they work.
3. Find all employees and their managers in the database who live in the same cities and on the same street number as do their managers.
4. Find all employees who earn more than the average salary of all employees of their company.
5. Find the company that pay least total salary along with the salary paid.
6. Give all managers of SBI a 10 percent raise.
7. Find the company that has the most employees
8. Find those companies whose employees earn a higher salary, on average than the average salary at Indian Bank.
9. Query to find name and salary of all employees who earn more than each employee of 'Indian Bank'

SQL QUERY

Company Table

company_name	city
SBI	Kochi
Indian Bank	Chennai
HDFC	Kochi

Employees Table

emp_id	emp_name	street	city	salary	company_name	manager_id
1	Arjun	MG Road	Kochi	50000	SBI	3
2	Neha	Anna St	Chennai	45000	Indian Bank	3
3	Manoj	MG Road	Kochi	70000	SBI	NULL
4	Riya	MG Road	Kochi	30000	HDFC	3
5	Kiran	Anna St	Chennai	28000	Indian Bank	2
6	Meera	MG Road	Kochi	55000	SBI	3

- Find the names of all employees who work for SBI.

```
SELECT emp_name
FROM Employees
WHERE company_name = 'SBI';
```

emp_name

Arjun

Manoj

Meera

- Find all employees in the database who live in the same cities as the companies for which they work.

```
SELECT e.emp_name, e.city, e.company_name
FROM Employees e
JOIN Company c ON e.company_name = c.company_name
WHERE e.city = c.city;
```

emp_name	city	company_name
Arjun	Kochi	SBI
Manoj	Kochi	SBI
Meera	Kochi	SBI
Neha	Chennai	Indian Bank
Kiran	Chennai	Indian Bank

3. Find all employees and their managers in the database who live in the same cities and on the same street number as do their managers.

```
SELECT e.emp_name, m.emp_name AS manager_name
FROM Employees e
JOIN Employees m ON e.manager_id = m.emp_id
WHERE e.city = m.city AND e.street = m.street;
```

emp_name	manager_name
Arjun	Manoj
Riya	Manoj
Meera	Manoj

4. Find all employees who earn more than the average salary of all employees of their company.

```
SELECT e.emp_name, e.salary, e.company_name
FROM Employees e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM Employees
    WHERE company_name = e.company_name
);
```

emp_name	salary	company_name
Manoj	70000	SBI
Neha	45000	Indian Bank

5. Find the company that pays the least total salary along with the salary paid.

```
SELECT company_name, SUM(salary) AS total_salary
FROM Employees
GROUP BY company_name
ORDER BY total_salary ASC
LIMIT 1;
```

company_name	total_salary
HDFC	30000

6. Give all managers of SBI a 10 percent raise.

```
UPDATE Employees
SET salary = salary * 1.10
WHERE emp_id IN (
    SELECT DISTINCT manager_id
    FROM Employees
    WHERE company_name = 'SBI'
);
```

emp_id	emp_name	salary
3	Manoj	77000

7. Find the company that has the most employees

```
SELECT company_name, COUNT(*) AS total_employees
FROM Employees
GROUP BY company_name
ORDER BY total_employees DESC
LIMIT 1;
```

company_name	total_employees
SBI	3

8. Find those companies whose employees earn a higher salary, on average than the average salary at Indian Bank.

```
SELECT company_name
FROM Employees
GROUP BY company_name
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM Employees
    WHERE company_name = 'Indian Bank')
```

```
);
```

```
company_name
```

```
SBI
```

9. Query to find name and salary of all employees who earn more than each employee of 'Indian Bank'

```
SELECT emp_name, salary
FROM Employees
WHERE salary > ALL (
    SELECT salary
    FROM Employees
    WHERE company_name = 'Indian Bank'
);
```

emp_name	salary
Manoj	70000
Meera	55000

MODEL VIVA QUESTIONS

1. What is the difference between DDL and DML?
2. Write any two DML commands.
3. What is the purpose of the WHERE clause?
4. Can we update multiple rows at the same time?
5. What happens if you delete a row without a WHERE clause?

RESULT:

The student successfully practiced various SQL DML commands including insertion, updation, deletion, altering data, and retrieving records based on conditions. The experiment helped in understanding how data is stored, modified, and queried in a relational database.

EXP NO:6**IMPLEMENTATION OF BUILT-IN FUNCTIONS IN RDBMS****AIM**

To study and implement various built-in functions available in an RDBMS using SQL queries.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Understand the use of built-in functions in SQL
- Apply numeric, string, date, and aggregate functions
- Retrieve and manipulate data efficiently using SQL functions

DESCRIPTION

Built-in functions in RDBMS are predefined functions that perform operations on data stored in tables. These functions simplify complex operations such as calculations, string manipulation, date processing, and summarization of data. Built-in functions are broadly classified into:

1. Numeric Functions – Perform mathematical operations
(e.g., ABS(), ROUND(), MOD())
2. String Functions – Manipulate character data
(e.g., UPPER(), LOWER(), LENGTH(), SUBSTR())
3. Date Functions – Handle date and time values
(e.g., SYSDATE, CURRENT_DATE)
4. Aggregate Functions – Perform calculations on a set of values
(e.g., SUM(), AVG(), COUNT(), MAX(), MIN())

SQL QUERY QUESTIONS

1. Display the absolute value of a given number.
2. Convert employee names to uppercase.
3. Find the length of a given string.
4. Display the current system date.
5. Find the total, average, maximum, and minimum salary of employees.

SQL QUERY

6. Display the absolute value of a given number.

```
SELECT ABS(-25) AS Absolute_Value;
```

```
+-----+
| Absolute_Value |
+-----+
| 25             |
+-----+
```

7. Convert employee names to uppercase.

```
SELECT UPPER(emp_name) AS Upper_Name FROM EMPLOYEE;
```

```
+-----+
| Upper_Name |
+-----+
| ANU        |
| RAHUL      |
| MEERA      |
+-----+
```

8. Find the length of a given string.

```
SELECT LENGTH(emp_name) AS Name_Length FROM EMPLOYEE;
```

```
+-----+
| Name_Length |
+-----+
| 3           |
| 5           |
| 5           |
+-----+
```

9. Display the current system date.

```
SELECT SYSDATE FROM DUAL;
```

```
+-----+
| SYSDATE      |
+-----+
| 11-MAR-2026  |
+-----+
```

10. Find the total, average, maximum, and minimum salary of employees

```
SELECT SUM(salary) AS Total_Salary FROM EMPLOYEE;
```

```

+-----+
| Total_Salary |
+-----+
| 90000       |
+-----+

```

SELECT AVG(salary) AS Average_Salary FROM EMPLOYEE;

```

+-----+
| Average_Salary |
+-----+
| 30000         |
+-----+

```

SELECT MAX(salary) AS Maximum_Salary FROM EMPLOYEE;

```

+-----+
| Maximum_Salary |
+-----+
| 35000         |
+-----+

```

SELECT MIN(salary) AS Minimum_Salary FROM EMPLOYEE;

```

+-----+
| Minimum_Salary |
+-----+
| 25000         |
+-----+

```

MODEL VIVA QUESTIONS

1. What are built-in functions in RDBMS?
2. Name the different types of built-in functions.
3. What is the use of aggregate functions?
4. What is the difference between COUNT(*) and COUNT(column_name)?
5. Give examples of any two string functions.

RESULT

Thus, the built-in functions in RDBMS were successfully implemented using SQL queries.

EXP NO:7**IMPLEMENTATION OF VARIOUS AGGREGATE FUNCTIONS IN SQL****AIM**

To study and implement various aggregate functions in SQL such as COUNT, SUM, AVG, MIN, and MAX for data analysis and summarization

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Use aggregate functions to perform calculations on table data
- Extract summarized information from a database
- Analyze numerical and non-numerical data efficiently
- Write accurate SQL queries for real-time database applications

DESCRIPTION

Aggregate functions in SQL are used to perform calculations on a set of values and return a single summarized result. These functions are commonly used in data analysis and reporting.

Common Aggregate Functions:

- COUNT() – Counts the number of rows
- SUM() – Calculates the total of values
- AVG() – Calculates the average value
- MIN() – Finds the minimum value
- MAX() – Finds the maximum value

Aggregate functions ignore NULL values except COUNT(*).

SQL QUERY QUESTIONS

6. Create a table named Employee.
7. Insert suitable records into the table.
8. Find the total number of employees.
9. Find the total salary of all employees.

10. Find the average salary.
11. Find the minimum and maximum salary.
12. Display the number of employees in each department.

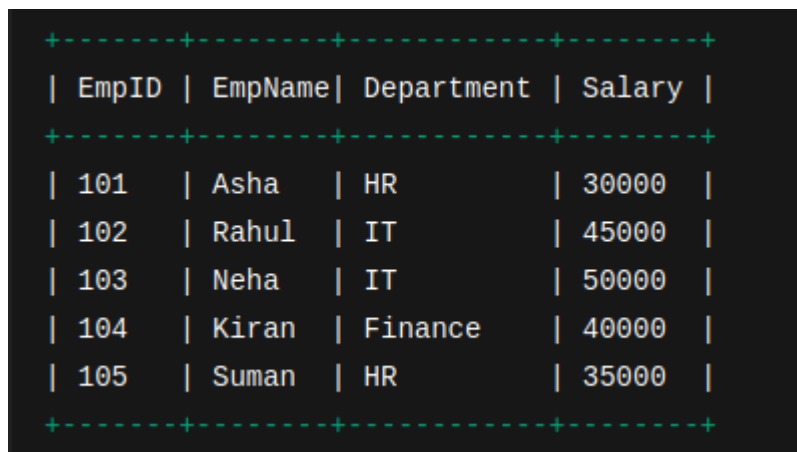
SQL QUERY

1. Create a table named Employee.

```
CREATE TABLE Employee ( EmpID INT PRIMARY KEY, EmpName VARCHAR(30),  
Department VARCHAR(20),Salary INT);
```

2. Insert suitable records into the table.

```
INSERT INTO Employee VALUES  
(101, 'Asha', 'HR', 30000),  
(102, 'Rahul', 'IT', 45000),  
(103, 'Neha', 'IT', 50000),  
(104, 'Kiran', 'Finance', 40000),  
(105, 'Suman', 'HR', 35000);  
SELECT * FROM Employee;
```



EmpID	EmpName	Department	Salary
101	Asha	HR	30000
102	Rahul	IT	45000
103	Neha	IT	50000
104	Kiran	Finance	40000
105	Suman	HR	35000

3. Find the total number of employees.

```
SELECT COUNT(*) AS Total_Employees FROM Employee;
```

```
+-----+
| Total_Employees |
+-----+
| 5               |
+-----+
```

4. Find the total salary of all employees.

```
SELECT SUM(Salary) AS Total_Salary FROM Employee;
```

```
+-----+
| Total_Salary |
+-----+
| 200000       |
+-----+
```

5. Find the average salary.

```
SELECT AVG(Salary) AS Average_Salary FROM Employee;
```

```
+-----+
| Average_Salary |
+-----+
| 40000          |
+-----+
```

6. Find the minimum and maximum salary.

```
SELECT MIN(Salary) AS Minimum_Salary FROM Employee;
```

```
+-----+
| Minimum_Salary |
+-----+
| 30000          |
+-----+
```

7. Display the number of employees in each department.

```
SELECT MAX(Salary) AS Maximum_Salary FROM Employee;
```

```
+-----+
| Maximum_Salary |
+-----+
| 50000          |
+-----+
```

MODEL VIVA QUESTIONS

1. What is ORDER BY clause?
2. What is a GROUP BY clause?
3. What is the HAVING clause?
4. Difference between WHERE and HAVING?
5. Can HAVING be used without GROUP BY?

RESULT:

All the SQL commands and queries were executed successfully, and the required outputs were obtained as per the objectives of the experiments.

EXP NO:8**IMPLEMENTATION OF ORDER BY, GROUP BY AND HAVING CLAUSE IN SQL****AIM**

To study and implement the use of ORDER BY, GROUP BY, and HAVING clauses in SQL for sorting, grouping, and filtering summarized data.

LEARNING OUTCOMES

After completing this experiment, you will be able to:

- Sort records using ORDER BY
- Group data using GROUP BY
- Filter grouped data using HAVING
- Use aggregate functions with grouping for data analysis

DESCRIPTION

ORDER BY - The ORDER BY clause is used to sort the result-set in ascending (ASC) or descending (DESC) order. By default, it sorts records in ascending order.

GROUP BY - The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country." It is often used with aggregate functions such as COUNT(), MAX(), MIN(), SUM(), and AVG().

HAVING - The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions. It allows us to filter the results returned by a GROUP BY clause based on a specified condition.

The standard order of execution is:

9. FROM (Identify tables)
10. WHERE (Filter raw rows)
11. GROUP BY (Aggregate rows)
12. HAVING (Filter groups)
13. SELECT (Choose columns)
14. ORDER BY (Sort final output)

SQL QUERY QUESTIONS

1. Display employees sorted by salary in ascending order.
2. Display employees sorted by age in descending order.
3. Count the number of employees in each department.
4. Find the average salary of each department.
5. Display departments where the average salary is greater than 40000.

SQL QUERY

1. Display employees sorted by salary in ascending order.

```
SELECT * FROM Employees ORDER BY Salary ASC;
```

name	department	salary
-----	-----	-----
Arjun Das	HR	28000
Sunita Rao	HR	32000
Anil Kumar	IT	45000
Ravi Verma	IT	55000
Priya Nair	Finance	60000

2. Display employees sorted by age in descending order.

```
SELECT * FROM Employees ORDER BY Age DESC;
```

name	department	age
-----	-----	-----
Arjun Das	HR	45
Priya Nair	Finance	40
Sunita Rao	HR	34
Anil Kumar	IT	28
Ravi Verma	IT	25

3. Count the number of employees in each department.

```
SELECT Department, COUNT(*) AS Total_Employees FROM Employees GROUP BY
Department;
```

department	total_employees
-----	-----
Finance	1
HR	2
IT	2

4. Find the average salary of each department.

```
SELECT Department, AVG(Salary) AS Avg_Salary FROM Employees GROUP BY
Department;
```

department	avg_salary
-----	-----
Finance	60000.0
HR	30000.0
IT	50000.0

5. Display departments where the average salary is greater than 40000.

```
SELECT Department, AVG(Salary) AS Avg_Salary FROM Employees GROUP BY
Department HAVING AVG(Salary) > 40000;
```

department	avg_salary
-----	-----
Finance	60000.0
IT	50000.0

MODEL VIVA QUESTIONS

1. What is the purpose of the ORDER BY clause?
2. What is the difference between WHERE and HAVING?
3. Why is GROUP BY used with aggregate functions?
4. Can HAVING be used without GROUP BY?
5. What is the default sorting order in ORDER BY?

RESULT

All the SQL commands and queries were executed successfully, and the required outputs were obtained as per the objectives of the experiments.

EXP NO:9**IMPLEMENTATION OF SET OPERATORS, NESTED QUERIES, AND JOIN QUERIES****AIM**

To practice and implement SQL queries using set operators (UNION, INTERSECT, MINUS), nested queries, and join queries to retrieve data from multiple tables effectively.

LEARNING OUTCOMES

After performing this experiment, students will be able to:

1. Understand and implement SQL set operators (UNION, INTERSECT, MINUS) to manipulate datasets.
2. Write and execute nested queries (subqueries) for complex data retrieval.
3. Understand and apply different join operations (INNER JOIN, LEFT JOIN, RIGHT JOIN) to combine data from multiple tables.
4. Analyze query results and understand the relationship between tables in a relational database.
5. Enhance SQL skills for real-world database querying and data manipulation.

DESCRIPTION

SET OPERATORS - Set operators in SQL are used to combine the results of two or more queries. The main set operators are UNION, INTERSECT, and MINUS/EXCEPT.

Syntax:

-- UNION: Combines results and returns distinct rows

```
SELECT column1, column2, ... FROM table1 UNION SELECT column1, column2, ... FROM table2;
```

-- INTERSECT: Returns only common rows

```
SELECT column1, column2, ... FROM table1 INTERSECT SELECT column1, column2, ... FROM table2;
```

-- MINUS / EXCEPT: Returns rows from first query not in second

```
SELECT column1, column2, ... FROM table1 MINUS SELECT column1, column2, ... FROM table2;
```

NESTED QUERIES (SUBQUERIES):

A nested query is a query inside another query. It is used to provide results to the outer query.

- Can be in SELECT, WHERE, or FROM clause.
- Helps in filtering, aggregation, and conditional retrieval.

Syntax:

-- Nested query in WHERE clause

```
SELECT column1, column2 FROM table1 WHERE column3 IN (SELECT column3 FROM table2 WHERE condition);
```

-- Nested query in SELECT clause

```
SELECT column1, (SELECT MAX(column2) FROM table2) AS max_val FROM table1;
```

3. JOIN QUERIES:

Joins combine rows from two or more tables based on a related column.

- INNER JOIN: Returns only matching rows.
- LEFT JOIN: Returns all rows from the left table, even if there is no match in the right table.
- RIGHT JOIN: Returns all rows from the right table.
- FULL OUTER JOIN: Returns all rows when there is a match or not.

Syntax:

-- INNER JOIN: Returns only matching rows

```
SELECT t1.column1, t2.column2 FROM table1 t1 INNER JOIN table2 t2 ON t1.key = t2.key;
```

-- LEFT JOIN: Returns all rows from left table, even if no match

```
SELECT t1.column1, t2.column2 FROM table1 t1 LEFT JOIN table2 t2 ON t1.key = t2.key;
```

-- RIGHT JOIN: Returns all rows from right table

```
SELECT t1.column1, t2.column2 FROM table1 t1 RIGHT JOIN table2 t2 ON t1.key = t2.key;
```

-- FULL OUTER JOIN: Returns all rows, matching or not

```
SELECT t1.column1, t2.column2 FROM table1 t1 FULL OUTER JOIN table2 t2 ON t1.key = t2.key;
```

SQL QUERY QUESTIONS

1. Create two tables Employee1 and Employee2 with fields: emp_id, emp_name, salary.
2. Write SQL queries using set operators to:

- a. Find all employees from both tables (UNION).
 - b. Find common employees in both tables (INTERSECT).
 - c. Find employees in Employee1 but not in Employee2 (MINUS).
3. Create a table Department with fields: dept_id, dept_name.
4. Create a table EmployeeDept with fields: emp_id, dept_id.
5. Write nested queries to:
 - a. List employees working in a specific department.
 - b. Find employees with salaries greater than the average salary.
6. Write join queries to:
 - a. Display employee details along with their department names using INNER JOIN.
 - b. Display all departments with employees (including departments with no employees) using LEFT JOIN.

SQL QUERY:

1. Create two tables Employee1 and Employee2 with fields: emp_id, emp_name, salary.

```
CREATE TABLE Employee1 ( emp_id INT PRIMARY KEY, emp_name VARCHAR(20),
salary DECIMAL(10,2));
```

```
INSERT INTO Employee1 VALUES (1,'Alice',45000);
```

```
INSERT INTO Employee1 VALUES (2,'Bob',50000);
```

```
INSERT INTO Employee1 VALUES (3,'Charlie',47000);
```

```
CREATE TABLE Employee2 ( emp_id INT PRIMARY KEY, emp_name VARCHAR(20),
salary DECIMAL(10,2));
```

```
INSERT INTO Employee2 VALUES (2,'Bob',50000);
```

```
INSERT INTO Employee2 VALUES (3,'Charlie',47000);
```

```
SELECT * FROM Employee1;
```

emp_id	emp_name	salary
1	Alice	45000.00
2	Bob	50000.00
3	Charlie	47000.00

```
SELECT * FROM Employee2;
```

emp_id	emp_name	salary
2	Bob	50000.00
3	Charlie	47000.00

2. Write SQL queries using set operators to:

- a. Find all employees from both tables (UNION).

```
SELECT emp_name FROM Employee1  
  
UNION  
  
SELECT emp_name FROM Employee2;
```

emp_name

Alice

Bob

Charlie

- b. Find common employees in both tables (INTERSECT).

```
SELECT emp_name FROM Employee1  
  
INTERSECT  
  
SELECT emp_name FROM Employee2;
```

emp_name

Bob

Charlie

- c. Find employees in Employee1 but not in Employee2 (MINUS).

```
SELECT emp_name FROM Employee1  
  
MINUS
```

```
SELECT emp_name FROM Employee2;
```

emp_name
Alice

3. Create a table Department with fields: dept_id, dept_name.

```
CREATE TABLE Department (dept_id INT PRIMARY KEY,  
dept_name VARCHAR(20));
```

```
INSERT INTO Department VALUES (101,'HR');
```

```
INSERT INTO Department VALUES (102,'IT');
```

```
INSERT INTO Department VALUES (103,'Finance');
```

dept_id	dept_name
101	HR
102	IT
103	Finance

4. Create a table EmployeeDept with fields: emp_id, dept_id.

```
CREATE TABLE EmployeeDept (emp_id INT,dept_id INT,  
FOREIGN KEY (emp_id) REFERENCES Employee1(emp_id),  
FOREIGN KEY (dept_id) REFERENCES Department(dept_id));
```

```
INSERT INTO EmployeeDept VALUES (1,101);
```

```
INSERT INTO EmployeeDept VALUES (2,102);
```

```
INSERT INTO EmployeeDept VALUES (3,102);
```

emp_id	dept_id
1	101
2	102
3	102

5. Write nested queries to:

- a. List employees working in a specific department.

```
SELECT emp_name FROM Employee1 WHERE emp_id IN ( SELECT emp_id
FROM EmployeeDept WHERE dept_id = 102);
```

- b. Find employees with salaries greater than the average salary.

```
SELECT emp_name, salary FROM Employee1 WHERE salary > (SELECT
AVG(salary) FROM Employee1);
```

```
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul    | 30000  |
| Meera    | 35000  |
+-----+-----+
```

6. Write join queries to:

- a. Display employee details along with their department names using INNER JOIN.

```
SELECT e.emp_name, d.dept_name FROM EmployeeDept ed INNER JOIN
Employee1 e ON ed.emp_id = e.emp_id INNER JOIN Department d ON ed.dept_id =
d.dept_id;
```

```
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Anu      | HR        |
| Rahul    | IT        |
| Meera    | IT        |
+-----+-----+
```

- b. Display all departments with employees (including departments with no employees) using LEFT JOIN.

```
SELECT d.dept_name, e.emp_name FROM Department d LEFT JOIN EmployeeDept
```

ed ON d.dept_id = ed.dept_id LEFT JOIN Employee1 e ON ed.emp_id = e.emp_id;

dept_name	emp_name
HR	Arjun
HR	Neha
Finance	NULL
IT	Meera
Marketing	NULL

MODEL VIVA QUESTIONS

1. What is a set operator in SQL? Name the types of set operators.
2. Can you update or delete data using a view?
3. What is the difference between MINUS (or EXCEPT) and INTERSECT?
4. When would you use a subquery instead of a join?
5. What happens if the tables in a join have no matching records?

RESULT

All SQL queries executed successfully using set operators, nested queries, and joins.

The outputs accurately show unions, intersections, differences, and employee–department relationships.

EXP NO:10**IMPLEMENTATION OF QUERIES USING TEMP TABLES****AIM**

To understand and implement Temporary Tables in SQL for storing intermediate results and simplifying complex multi-step queries.

LEARNING OUTCOMES

By the end of this implementation, you will be able to:

- Differentiate between Local and Global temporary tables.
- Use temp tables to break down complex logic into manageable steps.
- Manage database memory efficiently by utilizing tables that exist only for the duration of a session.
- Perform CRUD operations on temporary datasets.

DESCRIPTION

A Temporary Table is used to store data temporarily during a database session.

It is automatically deleted when the session ends.

Types (SQL Server style):

- #TempTable → Local temporary table (visible only in current session)
- ##TempTable → Global temporary table (visible to all sessions)

Temp tables are useful when:

- Breaking complex queries into steps
- Storing intermediate results
- Improving performance and readability

SQL QUERY QUESTIONS

1. Create a local temporary table for storing employee details.
2. Insert at least five records into the temporary table.
3. Display all records from the temporary table.
4. Display employees whose salary is greater than 30000.

5. Display employees working in the IT department.
6. Update the salary of an employee in the temporary table.
7. Delete a record from the temporary table.
8. Create a global temporary table and insert sample data.
9. Count the number of employees stored in the temporary table.
10. Drop the temporary table.

SQL QUERY

1. Create a local temporary table for storing employee details.

```
CREATE TABLE #Employee (EmpID INT, EmpName VARCHAR(50), Department
VARCHAR(50), Salary INT);
```

2. Insert at least five records into the temporary table.

```
INSERT INTO #Employee VALUES(1,'Anil','HR',25000), (2,'Ravi','IT',32000),
(3,'Meena','Finance',28000), (4,'Suresh','IT',40000), (5,'Latha','HR',26000);
```

3. Display all records from the temporary table.

```
SELECT * FROM #Employee;
```

EmpID	EmpName	Department	Salary
1	Anil	HR	25000
2	Ravi	IT	32000
3	Meena	Finance	28000
4	Suresh	IT	40000
5	Latha	HR	26000

4. Display employees whose salary is greater than 30000.

```
SELECT * FROM #Employee
```

```
WHERE Salary > 30000;
```

EmpID	EmpName	Department	Salary
2	Ravi	IT	32000
4	Suresh	IT	40000

5. Display employees working in the IT department.

```
SELECT * FROM #Employee WHERE Department='IT';
```

EmpID	EmpName	Department	Salary
2	Ravi	IT	32000
4	Suresh	IT	40000

6. Update the salary of an employee in the temporary table.

```
UPDATE #Employee SET Salary = 35000 WHERE EmpID = 1;
```

EmpID	EmpName	Department	Salary
1	Anil	HR	35000
2	Ravi	IT	32000
3	Meena	Finance	28000
4	Suresh	IT	40000
5	Latha	HR	26000

7. Delete a record from the temporary table.

```
DELETE FROM #Employee WHERE EmpID = 5;
```

EmpID	EmpName	Department	Salary
1	Anil	HR	35000
2	Ravi	IT	32000
3	Meena	Finance	28000
4	Suresh	IT	40000

8. Create a global temporary table and insert sample data.

```
CREATE TABLE ##Department (DeptID INT, DeptName VARCHAR(50));
```

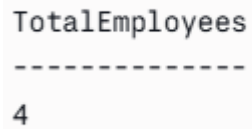
```
INSERT INTO ##Department VALUES(1,'HR'),(2,'IT'),(3,'Finance');
```

```
SELECT * FROM ##Department;
```

DeptID	DeptName
1	HR
2	IT
3	Finance

9. Count the number of employees stored in the temporary table.

```
SELECT COUNT(*) AS TotalEmployees FROM #Employee;
```



A screenshot of a SQL query result. It shows a single column header 'TotalEmployees' followed by a dashed line separator and the value '4'.

10. Drop the temporary table.

```
DROP TABLE #Employee;
```

MODEL VIVA QUESTIONS

12. What is a Temporary Table?
13. What is a Local Temporary Table?
14. What is a Global Temporary Table?
15. What symbol is used for Local Temporary Tables?
16. What symbol is used for Global Temporary Tables?

RESULT:

Temporary Tables were successfully created and employee records were inserted and displayed using SQL queries.

EXP NO:11**IMPLEMENTATION OF SQL TCL COMMANDS****AIM**

To practice and understand Transaction Control Language (TCL) commands such as COMMIT, ROLLBACK, and SAVEPOINT, which help in managing transactions and maintaining data integrity in a database.

LEARNING OUTCOMES

After performing this experiment, students will be able to:

1. Understand the concept of transactions in SQL.
2. Use COMMIT to permanently save changes made to a database.
3. Use ROLLBACK to undo unwanted changes.
4. Create and use SAVEPOINTS to partially roll back a transaction.

Ensure data consistency and error recovery using TCL commands.

DESCRIPTION

TRANSACTIONAL CONTROL LANGUAGE (T.C.L): A transaction is a logical unit of work. All changes made to the database can be referred to as a transaction. Transaction changes can be made permanent to the database only if they are committed; a transaction begins with an executable SQL statement & ends explicitly with either roll back or commit statement.

COMMIT: This command is used to end a transaction only with the help of the commit command transaction changes can be made permanent to the database.

Syntax: SQL>COMMIT;

Example: SQL>COMMIT;

SAVE POINT: Save points are like marks to divide a very lengthy transaction to smaller once. They are used to identify a point in a transaction to which we can later roll back. Thus, save points are used in conjunction with roll back.

Syntax: SQL>SAVE POINT ID;

ROLL BACK:

A roll back command is used to undo the current transactions. We can roll back the entire transaction so that all changes made by SQL statements are undo (or) roll back a transaction to a save point so that the SQL statements after the save point are rolled back.

Syntax:

ROLL BACK(current transaction can be roll back)

ROLL BACK to save point ID;

SAVE POINT:

Save points are like marks to divide a very lengthy transaction to smaller once. They are used to identify a point in a transaction to which we can later roll back. Thus, save point is used in conjunction with roll back.

Syntax:

SQL> SAVE POINT ID;

SQL QUERY QUESTIONS

1. Create a database NEW.
2. Create a table CLASS with the ID and Name of the student.
3. Insert 5 rows into the table.
4. Display the table.
5. Begin transaction and insert a new row into the table.
6. Create a savepoint.
7. Insert a new row into the table and display the table.
8. Rollback to savepoint and display the table.
9. Update the table with the new name of the student having ID=5 and display the table.
10. Rollback to savepoint and display the table.
11. Finally commit the transaction.

SQL COMMANDS:

- 1) Create a database NEW.

CREATE DATABASE NEW;

USE NEW;

- 2) Create a table CLASS with the ID and Name of the student.

```
CREATE TABLE CLASS (ID INT PRIMARY KEY, Name VARCHAR(50));
```

- 3) Insert 5 rows into the table.

```
INSERT INTO CLASS VALUES (1, 'Avani');
```

```
INSERT INTO CLASS VALUES (2, 'Sree');
```

```
INSERT INTO CLASS VALUES (3, 'Indra');
```

```
INSERT INTO CLASS VALUES (4, 'Nanda');
```

```
INSERT INTO CLASS VALUES (5, 'Bhoomi');
```

- 4) Display the table.

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi

- 5) Begin transaction and insert a new row into the table.

```
START TRANSACTION;
```

```
INSERT INTO CLASS VALUES (6, 'Krishna');
```

- 6) Create a savepoint.

```
SAVEPOINT SP1;
```

- 7) Insert a new row into the table and display the table.

```
INSERT INTO CLASS VALUES (7, 'Shiva');
```

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi
6	Krishna
7	Shiva

8) Rollback to savepoint and display the table.

```
ROLLBACK TO SP1;
```

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi
6	Krishna

9) Update the table with the new name of the student having ID=5 and display the table.

```
UPDATE CLASS SET Name = 'Shiva' WHERE ID = 5;
```

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Shiva
6	Krishna

10) Rollback to savepoint and display the table.

ROLLBACK TO SP1;

SELECT * FROM CLASS;

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi
6	Krishna

11) Finally commit the transaction.

COMMIT;

RESULT:

All the SQL commands and queries were executed successfully, and the required outputs were obtained as per the objectives of the experiments.

EXP NO:12**IMPLEMENTATION OF SQL DCL COMMANDS****AIM:**

To practice SQL Data Control Language (DCL) commands such as GRANT and REVOKE used to manage user privileges in a database.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand database security and access control.
2. Grant specific privileges to users on database objects.
3. Revoke assigned privileges as needed.
4. Differentiate between system and object privileges.
5. Implement controlled data access in a multi-user environment.

DESCRIPTION:

PRIVILEGES: A privilege is a right to execute an SQL statement or to access another user's object. In Oracle, there are two types of privileges

- System Privileges
- Object Privileges

System Privileges are those through which the user can manage the performance of database actions. It is normally granted by DBA to users. Eg: Create Session, Create Table, Create user etc..

Object Privileges allow access to objects or privileges on objects, i.e. tables, table columns, tables, views etc.. It includes alter, delete, insert, select update etc. (After creating the user, DBA grant specific system privileges to user)

GRANT: The DBA uses the GRANT statement to allocate system privileges to other users.

Syntax:

SQL> GRANT privilege [privilege....] TO USER ;

SQL> Grant succeeded

```
SQL> GRANT object_priv [(column)] ON object TO user;
```

```
SQL>GRANT select, insert ON emp TO James;
```

```
SQL>GRANT select ,update (e_name,e_address) ON emp TO James;
```

CHANGE PASSWORD: The DBA creates an account and initializes a password for every user. You can change password by using ALTER USER statement.

Syntax:

Alter USER IDENTIFIED BY

REVOKE: A REVOKE statement is used to remove privileges granted to other users. The privileges you specify are revoked from the users.

Syntax:

```
REVOKE [privilege.. ...] ON object FROM user
```

ROLE: A role is a named group of related privileges that can be granted to a user. In other words, role is a predefined collection of privileges that are grouped together, thus privileges are easier to assign users.

```
SQL> Create role custom;
```

```
SQL> Grant create table, create view TO custom;
```

```
SQL> Grant select, insert ON emp TO custom;
```

SQL QUERY QUESTIONS

- 1) Create a new user USER1 with a password.
- 2) Grant CREATE, SELECT and INSERT privileges to the new user, USER1.
- 3) Exit from the root user.
- 4) Login to the new user, USER1.
- 5) Insert a new row into the CLASS table of database NEW.
- 6) Display the table CLASS and Exit from USER1.
- 7) Login to the root user and REVOKE INSERT privileges of USER1 from the CLASS table of database NEW.
- 8) Exit from the root user.
- 9) Login to the new user, USER1 and insert a new row into the CLASS table of database NEW.

SQL COMMANDS:

- 1) Create a new user USER1 with a password.

```
CREATE USER 'USER1'@'localhost' IDENTIFIED BY 'password123';
```

```
CREATE DATABASE NEW;
```

```
USE NEW;
```

```
CREATE TABLE CLASS (ID INT PRIMARY KEY,Name VARCHAR(50));
```

```
INSERT INTO CLASS VALUES (1, 'Avani');
```

```
INSERT INTO CLASS VALUES (2, 'Sree');
```

```
INSERT INTO CLASS VALUES (3, 'Indra');
```

```
INSERT INTO CLASS VALUES (4, 'Nanda');
```

```
INSERT INTO CLASS VALUES (5, 'Bhoomi');
```

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi

- 2) Grant CREATE, SELECT and INSERT privileges to the new user, USER1.

```
GRANT CREATE, SELECT, INSERT ON *.* TO 'USER1'@'localhost';
```

```
FLUSH PRIVILEGES;
```

- 3) Exit from the root user.

```
EXIT;
```

- 4) Login to the new user, USER1.

```
mysql -u USER1 -p
```

- 5) Insert a new row into the CLASS table of database NEW.

```
USE NEW;
```

```
INSERT INTO CLASS (ID, Name) VALUES (6, 'Sujith');
```

- 6) Display the table CLASS and Exit from USER1.

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi
6	Sujith

```
EXIT;
```

- 7) Login to the root user and REVOKE INSERT privileges of USER1 from the CLASS table of database NEW.

```
REVOKE INSERT ON NEW.CLASS FROM 'USER1'@'localhost';
```

- 8) Exit from the root user.

```
EXIT;
```

- 9) Login to the new user, USER1 and insert a new row into the CLASS table of database NEW.

```
Login back in as USER1.
```

```
INSERT INTO CLASS (ID, Name) VALUES (2, 'Science');
```

MODEL VIVA QUESTIONS

1. What is the difference between DCL and TCL?
2. Give examples of DCL commands.
3. What is the use of the GRANT command?
4. What happens when privileges are revoked?
5. What is the difference between system and object privileges?

RESULT:

All the SQL commands and queries were executed successfully, and the required outputs were obtained as per the objectives of the experiments.

EXP NO:13**CREATION AND MANIPULATION OF TABLES, VIEWS, AND ASSERTIONS
IN SQL****AIM**

To practice SQL commands for the creation and management of Views and Assertions in a database system.

LEARNING OUTCOMES

After completing this practice, students will be able to:

1. Create Views: Understand how to create virtual tables (views) using SQL commands to simplify complex queries.
2. Manipulate Views: Learn how to select, update, and delete data using views.
3. Understand Assertions: Learn how to create assertions to enforce integrity constraints that cannot be captured by traditional primary and foreign key constraints.
4. Apply SQL: Effectively use SQL to manage data integrity and query optimization.

DESCRIPTION**1. Views (Virtual Tables)**

A View is a virtual table whose content is defined by a stored SQL query. It does not store data but rather pulls current data from one or more underlying base tables every time it is accessed.

- Purpose: To simplify complex queries (abstracting joins), enforce security (restricting access to certain columns/rows), and provide customized data presentation.
- Updatability: Simple views (derived from a single table and including its primary key) are generally updatable (allow INSERT, UPDATE, DELETE). Complex views (involving joins or aggregate functions) are often read-only.

Syntax:

```
CREATE VIEW ViewName AS SELECT column1, column2, ... FROM TableName(s)
```

```
WHERE condition;
```

Syntax for Deletion:

DROP VIEW ViewName;

2. Assertions (Database Constraints):

An Assertion defines a condition that the database must always maintain across its state. It is a powerful form of integrity constraint that can involve multiple tables or complex calculations.

- **Purpose:** To enforce complex business rules that go beyond simple column- or row-level constraints (like ensuring the total count of 'x' in Table A always matches the count of 'y' in Table B).
- **Status in RDBMS:** While defined in standard SQL, Assertions are typically not supported in modern relational database systems (like MySQL, PostgreSQL, or SQL Server). Their function is usually implemented using Triggers or complex Stored Procedures.

Syntax:

CREATE ASSERTION AssertionName CHECK (Condition);

Example:

CREATE ASSERTION check_fee_positive CHECK

(NOT EXISTS (SELECT * FROM ACCOUNT_DETAILS WHERE sem_fee <= 0));

SQL QUERY QUESTIONS

1. Create two tables STUDENT_DETAILS and ACCOUNT_DETAILS with the following columns:

STUDENT_DETAILS:

- std_id
- std_name
- branch

ACCOUNT_DETAILS:

- std_id
- std_name
- sem_fee

2. Create a view VIEW1 that contains std_id and branch.

3. Create a view VIEW2 that contains std_id, std_name, sem_fee, and branch.
4. Perform UPDATE and DELETE operations in the created views.
5. Insert a new row into VIEW1.
6. Drop the view.
7. Create an assertion that enforces the rule that std_id in STUDENT_DETAILS is always NOT NULL, since each student must have one std_id.

SQL COMMANDS:

1. Table 1 STUDENT_DETAILS

```
CREATE TABLE STUDENT_DETAILS (std_id INT PRIMARY KEY, std_name  
VARCHAR(100) NOT NULL, branch VARCHAR(50));
```

Table 2: ACCOUNT_DETAILS

```
CREATE TABLE ACCOUNT_DETAILS ( std_id INT PRIMARY KEY,sem_fee  
DECIMAL(10, 2),
```

```
FOREIGN KEY (std_id) REFERENCES STUDENT_DETAILS(std_id));
```

```
INSERT INTO STUDENT_DETAILS (std_id, std_name, branch) VALUES
```

```
(1, 'Alice', 'CSE'),(2, 'Bob', 'ECE'),(3, 'Charlie', 'CSE');
```

```
INSERT INTO ACCOUNT_DETAILS (std_id, sem_fee) VALUES
```

```
(1, 45000.00),(2, 50000.00),(3, 45000.00);
```

2. Create a view VIEW1 that contains std_id and branch.

```
CREATE VIEW view1 AS SELECT std_id, branch FROM student_details;
```

```
SELECT * FROM VIEW1;
```

std_id	branch
1	CSE
2	ECE
3	CSE

3. Create a view VIEW2 that contains std_id, std_name, sem_fee, and branch.

```
CREATE VIEW view2 AS
SELECT s.std_id, s.std_name, a.sem_fee, s.branch
FROM student_details s
JOIN account_details a
ON s.std_id = a.std_id;
```

```
SELECT * FROM VIEW2;
```

std_id	std_name	sem_fee	branch
1	Alice	45000.00	CSE
2	Bob	50000.00	ECE
3	Charlie	45000.00	CSE

4. Perform UPDATE and DELETE operations in the created views.

```
UPDATE view2 SET sem_fee = sem_fee + 1000 WHERE std_id = 1;
```

std_id	std_name	sem_fee	branch
1	Alice	46000.00	CSE
2	Bob	50000.00	ECE
3	Charlie	45000.00	CSE

```
DELETE FROM view2 WHERE std_id = 2;
```

std_id	std_name	sem_fee	branch
1	Alice	46000.00	CSE
3	Charlie	45000.00	CSE

5. Insert a new row into VIEW1.

```
INSERT INTO view1 VALUES (4, 'MECH');
```

std_id	branch
1	CSE
2	ECE
3	CSE
4	MECH

6. Drop the view.

```
DROP VIEW view1;
```

```
DROP VIEW view2;
```

7. Create an assertion that enforces the rule that std_id in STUDENT_DETAILS is always NOT NULL, since each student must have one std_id.

```
ALTER TABLE STUDENT_DETAILS MODIFY std_id NOT NULL;
```

```
CREATE ASSERTION std_id_not_null
```

```
CHECK ( NOT EXISTS (SELECT * FROM STUDENT_DETAILS WHERE std_id IS NULL));
```

```
SQL> ALTER TABLE STUDENT_DETAILS MODIFY std_id NOT NULL;
```

```
Table altered.
```

```
SQL> CREATE ASSERTION std_id_not_null
```

```
2 CHECK (NOT EXISTS
```

```
3 (SELECT * FROM STUDENT_DETAILS WHERE std_id IS NULL));
```

```
Assertion created.
```

MODEL VIVA QUESTIONS

3. What is a View in SQL, and how is it different from a table?
4. Can you update data through a View? If yes, under what conditions?
5. How would you enforce a condition like "age cannot be less than 18" for every row in a table?
6. What is the purpose of an assertion in SQL?
7. What is the difference between a VIEW and a MATERIALIZED VIEW

RESULT:

The SQL queries were executed successfully, and the required outputs for table creation, views, view manipulation, and assertion were obtained.

EXP NO:14**IMPLEMENTATION OF CONTROL STRUCTURES IN PL/SQL****AIM**

To study and implement various control structures like IF-THEN, IF-THEN-ELSE, IF-THEN-ELSIF, CASE and WHILE in PL/SQL

LEARNING OUTCOMES

- Understand PL/SQL control statements.
- Implement decision-making statements.
- Use looping structures.
- Develop PL/SQL programs.

DESCRIPTION

PL/SQL provides several control structures that allow programmers to control the flow of execution in a program. These structures help in making decisions, performing repetitive tasks, and executing specific blocks of code based on conditions. The commonly used control structures in PL/SQL include decision-making statements such as IF-THEN, IF-THEN-ELSE, IF-THEN-ELSIF, and CASE, as well as looping structures like WHILE.

The IF-THEN statement is used to execute a block of code only when a specified condition is true. The IF-THEN-ELSE statement provides an alternative block of code that executes when the condition is false. The IF-THEN-ELSIF structure allows checking multiple conditions sequentially. The CASE statement simplifies complex conditional logic by selecting actions based on the value of an expression.

Looping structures such as the WHILE loop allow repeated execution of a block of statements as long as a specified condition remains true. These control structures are essential for developing efficient and dynamic PL/SQL programs.

ALGORITHM

1. Start the program.
2. Declare variables.
3. Assign values.

4. Use IF and CASE statements.
5. Use WHILE loop.
6. Display output.
7. Stop.

PROGRAM

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
num NUMBER:=10;
```

```
i NUMBER:=1;
```

```
grade CHAR:='A';
```

```
BEGIN
```

```
-- IF THEN
```

```
IF num > 5 THEN
```

```
DBMS_OUTPUT.PUT_LINE('Number is greater than 5');
```

```
END IF;
```

```
-- IF THEN ELSE
```

```
IF num MOD 2 = 0 THEN
```

```
DBMS_OUTPUT.PUT_LINE('Even Number');
```

```
ELSE
```

```
DBMS_OUTPUT.PUT_LINE('Odd Number');
```

```
END IF;
```

```
-- IF THEN ELSIF
```

```
IF num < 5 THEN
```

```
DBMS_OUTPUT.PUT_LINE('Less than 5');
```

```
ELSIF num = 10 THEN

DBMS_OUTPUT.PUT_LINE('Equal to 10');

ELSE

DBMS_OUTPUT.PUT_LINE('Greater than 10');

END IF;

-- CASE

CASE grade

WHEN 'A' THEN

DBMS_OUTPUT.PUT_LINE('Excellent');

WHEN 'B' THEN

DBMS_OUTPUT.PUT_LINE('Good');

ELSE

DBMS_OUTPUT.PUT_LINE('Average');

END CASE;

-- WHILE LOOP

WHILE i<=5 LOOP

DBMS_OUTPUT.PUT_LINE(i);

i:=i+1;

END LOOP;

END;

/
```

OUTPUT

```
Number is greater than 5  
Even Number  
Equal to 10  
Excellent  
1  
2  
3  
4  
5  
  
PL/SQL procedure successfully completed.
```

MODEL VIVA QUESTIONS

1. What is PL/SQL?
2. What is the IF statement?
3. What is the CASE statement?
4. What is the WHILE loop?
5. What is DBMS_OUTPUT?

RESULT:

Thus various control structures were successfully implemented using PL/SQL.

EXP NO: 15**CREATION OF PROCEDURES, TRIGGERS AND FUNCTIONS****AIM**

To create Procedures, Triggers and Functions using PL/SQL.

LEARNING OUTCOMES

- Understand procedures.
- Understand triggers.
- Understand functions.
- Execute PL/SQL blocks.

DESCRIPTION

PL/SQL provides the ability to create reusable program units such as procedures, functions, and triggers to perform specific tasks in a database. A procedure is a named PL/SQL block that performs a particular action and can be executed multiple times, helping in modular programming and code reusability. The basic syntax of a procedure is:

syntax: CREATE OR REPLACE PROCEDURE procedure_name IS BEGIN -- statements END;

A function is similar to a procedure but it always returns a value to the calling program and is commonly used in calculations or queries. The basic syntax of a function is:

syntax: CREATE OR REPLACE FUNCTION function_name RETURN datatype IS BEGIN -- statements RETURN value; END;

A trigger is a special type of PL/SQL block that automatically executes when a specified database event such as INSERT, UPDATE, or DELETE occurs on a table. Triggers help maintain data integrity and enforce business rules. The basic syntax of a trigger is:

syntax: CREATE OR REPLACE TRIGGER trigger_name BEFORE/AFTER INSERT OR UPDATE OR DELETE ON table_name FOR EACH ROW BEGIN -- statements END;

ALGORITHM

1. Create a table.

2. Insert values.
3. Create a procedure.
4. Create a function.
5. Create triggers.
6. Execute program.
7. Stop.

PROGRAM

```
CREATE TABLE Student (ID NUMBER,Name VARCHAR2(20),Marks NUMBER);
```

```
INSERT INTO Student VALUES(1,'Anil',80);
```

```
INSERT INTO Student VALUES(2,'Ravi',70);
```

Procedure

```
CREATE OR REPLACE PROCEDURE display_student
```

```
IS
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('Procedure Executed');
```

```
END;
```

```
/
```

Function

```
CREATE OR REPLACE FUNCTION add_num(a NUMBER,b NUMBER)
```

```
RETURN NUMBER
```

```
IS
```

```
c NUMBER;
```

```
BEGIN
```

```
c:=a+b;
```

```
RETURN c;
```

END;

/

Trigger

CREATE OR REPLACE TRIGGER student_trigger

BEFORE INSERT ON Student

FOR EACH ROW

BEGIN

DBMS_OUTPUT.PUT_LINE('Record Inserted');

END;

/

OUTPUT

```
SQL> @student_program.sql
```

```
Table created.
```

```
1 row created.
```

```
1 row created.
```

```
Procedure created.
```

```
Procedure Executed  
PL/SQL procedure successfully completed.
```

```
Function created.
```

```
ADD_NUM(5,3)
```

```
-----
```

```
8
```

```
Trigger created.
```

```
Record Inserted  
1 row created.
```

MODEL VIVA QUESTIONS

1. What is Procedure?
2. What is Function?
3. What is Trigger?
4. When does the trigger work?
5. What is PL/SQL block?

RESULT

Thus procedures, triggers and functions were created successfully.

EXP NO: 16**CREATION OF PACKAGES****AIM**

To create packages using PL/SQL.

LEARNING OUTCOMES

- Understand packages.
- Create package specification.
- Create a package body.
- Execute package.

DESCRIPTION

PL/SQL Packages are used to group related procedures and functions into one unit. Packages help to organize programs and make them easy to use.

Packages are supported by Oracle Corporation in PL/SQL.

A package has two parts:

- Package Specification – Contains the declaration of procedures and functions.
- Package Body – Contains the actual program code.

ALGORITHM

1. Start.
2. Create package specification.
3. Create a package body.
4. Execute package.
5. Stop.

PROGRAM

– *Package Specification*

CREATE OR REPLACE PACKAGE math_pack

```
IS

FUNCTION add_num(a NUMBER,b NUMBER)

RETURN NUMBER;

PROCEDURE display;

END;

/

--Package Body
CREATE OR REPLACE PACKAGE BODY math_pack

IS

FUNCTION add_num(a NUMBER,b NUMBER)

RETURN NUMBER

IS

BEGIN

RETURN a+b;

END;

PROCEDURE display

IS

BEGIN

DBMS_OUTPUT.PUT_LINE('Package Executed');

END;

END;

/

Execute Package

SET SERVEROUTPUT ON;

BEGIN

math_pack.display;

END;
```

/

OUTPUT

```
SQL> @math_pack.sql

Package created.

Package body created.

SQL> SET SERVEROUTPUT ON;

SQL> BEGIN
  2  math_pack.display;
  3  END;
  4  /

Package Executed

PL/SQL procedure successfully completed.
```

MODEL VIVA QUESTIONS

1. What is a Package?
2. What is the Package specification?
3. What is a Package body?
4. Advantage of package?
5. What is PL/SQL?

RESULT

Thus packages were created and executed successfully.

EXP NO:17**CREATION OF CURSORS****AIM**

To create and implement cursors in PL/SQL to fetch and display records from a table.

LEARNING OUTCOMES

- Understand the concept of cursors in PL/SQL.
- Learn how to declare, open, fetch and close a cursor.
- Display records from a table using a cursor.

DESCRIPTION

A cursor is a pointer used in PL/SQL to retrieve rows from a query result one at a time.

There are two types of cursors:

- Implicit Cursor – Automatically created by Oracle.
- Explicit Cursor – Defined by the programmer to retrieve multiple rows.

Explicit cursors involve four steps:

1. Declare cursor
2. Open cursor
3. Fetch records
4. Close cursor

ALGORITHM

1. Start the program.
2. Create a student table and insert values.
3. Declare a cursor to select student records.
4. Open the cursor.
5. Fetch records one by one.
6. Display each record using DBMS_OUTPUT.
7. Close the cursor.

8. Stop.

PROGRAM

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
CURSOR c1 IS SELECT * FROM Student;
```

```
v_id Student.ID%TYPE;
```

```
v_name Student.Name%TYPE;
```

```
v_marks Student.Marks%TYPE;
```

```
BEGIN
```

```
OPEN c1;
```

```
LOOP
```

```
FETCH c1 INTO v_id,v_name,v_marks;
```

```
EXIT WHEN c1%NOTFOUND;
```

```
DBMS_OUTPUT.PUT_LINE(v_id||' '||v_name||' '||v_marks);
```

```
END LOOP;
```

```
CLOSE c1;
```

```
END;
```

```
/
```

OUTPUT

```
SQL> SET SERVEROUTPUT ON
```

```
SQL> /
```

```
1 Anil 80  
2 Ravi 70  
3 Rahul 85
```

```
PL/SQL procedure successfully completed.
```

MODEL VIVA QUESTIONS

1. What is a cursor in PL/SQL?
2. What are the types of cursors?
3. What are the steps involved in cursor execution?
4. What is the difference between implicit and explicit cursor?
5. What is the use of the%NOTFOUND attribute?

RESULT

Thus the cursor was successfully created and executed to fetch records from the table.

EXP NO:18**CREATION OF PL/SQL BLOCKS FOR EXCEPTION HANDLING****AIM**

To write a PL/SQL block to handle exceptions during program execution.

LEARNING OUTCOMES

- Understand runtime errors in PL/SQL.
- Learn how to use exception handling.
- Handle errors like division by zero.

DESCRIPTION

Exception handling is used in PL/SQL to handle runtime errors.

It prevents the program from crashing and allows the programmer to display meaningful messages.

Types of exceptions:

- Predefined exceptions (ZERO_DIVIDE, NO_DATA_FOUND)
- User-defined exceptions

The exception section begins with the keyword EXCEPTION.

ALGORITHM

1. Start the program.
2. Declare variables.
3. Perform a division operation.
4. If an error occurs, transfer control to the exception block.
5. Display the error message.
6. Stop the program.

PROGRAM

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
a NUMBER:=10;
```

```
b NUMBER:=0;

c NUMBER;

BEGIN

c:=a/b;

DBMS_OUTPUT.PUT_LINE('Result='||c);

EXCEPTION

WHEN ZERO_DIVIDE THEN

DBMS_OUTPUT.PUT_LINE('Error: Division by zero');

END;

/
```

OUTPUT

```
SQL> SET SERVEROUTPUT ON
SQL> /
```

```
Error: Division by zero
```

```
PL/SQL procedure successfully completed.
```

MODEL VIVA QUESTIONS

1. What is exception handling in PL/SQL?
2. What are predefined exceptions?
3. What is the ZERO_DIVIDE exception?
4. What is the purpose of the EXCEPTION block?
5. Can we create user-defined exceptions?

RESULT

Thus the PL/SQL block for exception handling was successfully executed.

EXP NO:19**DATABASE BACKUP AND RESTORE USING COMMANDS****AIM**

To perform database backup and restore operations using Oracle commands.

LEARNING OUTCOMES

- Understand the importance of database backup.
- Learn commands for backup and restore.
- Protect data from accidental loss.

DESCRIPTION

Database backup is the process of creating a copy of the database to prevent data loss.

Restore is the process of recovering the database from the backup.

Common backup types:

- Full backup
- Incremental backup

Oracle uses EXP (Export) and IMP (Import) utilities for backup and restore.

ALGORITHM

1. Start the command prompt.
2. Use the EXP command to export database objects.
3. Store the backup file.
4. Use the IMP command to restore the database.
5. Verify the restored data.
6. Stop.

COMMANDS

Backup Command : exp username/password file=backup.dmp

Restore Command : imp username/password file=backup.dmp

OUTPUT

```
C:\> exp system/oracle file=backup.dmp

Export: Release 11.2.0.2.0
Connected to: Oracle Database

Export done in WE8MSWIN1252 character set
. exporting table STUDENT

Export terminated successfully without warnings.
```

```
C:\> imp system/oracle file=backup.dmp

Import: Release 11.2.0.2.0
Connected to: Oracle Database

. importing table STUDENT

Import terminated successfully without warnings.
```

MODEL VIVA QUESTIONS

1. What is database backup?
2. What is the difference between backup and restore?
3. What is the EXP command?
4. What is the IMP command?
5. Why is database backup important?

RESULT

Thus the database backup and restore operations were successfully performed using Oracle commands.

EXP NO:20**QUERY ANALYSIS USING QUERY PLAN / SHOW PLAN****AIM**

To analyze the execution of SQL queries using Query Plan / Show Plan.

LEARNING OUTCOMES

- Understand how the database executes SQL queries.
- Learn to analyze query performance.
- Identify how indexes and tables are accessed.

DESCRIPTION

A Query Plan (Execution Plan) shows how a database engine executes a query. It displays the sequence of operations such as:

- Table scan
- Index scan
- Join operations
- Filtering

Using EXPLAIN PLAN helps optimize queries and improve database performance.

SQL QUERY QUESTIONS

1. Display all student records and analyze the query plan.
2. Analyze the query plan for selecting students with marks greater than 70
3. Analyze the query plan for selecting only student names.

SQL QUERY

1. Display all student records and analyze the query plan.

EXPLAIN PLAN FOR SELECT * FROM Student;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

```
SQL> EXPLAIN PLAN FOR
2  SELECT * FROM Student;

Explained.
```

```
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
PLAN_TABLE_OUTPUT
```

```
-----
| Id | Operation          | Name      |
-----
| 0  | SELECT STATEMENT   |           |
| 1  | TABLE ACCESS FULL | STUDENT   |
-----
```

2. Analyze the query plan for selecting students with marks greater than 70

```
EXPLAIN PLAN FOR SELECT * FROM Student WHERE Marks > 70;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
SQL> EXPLAIN PLAN FOR
2  SELECT * FROM Student WHERE Marks > 70;

Explained.
```

```
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
PLAN_TABLE_OUTPUT
```

```
-----
| Id | Operation          | Name      |
-----
| 0  | SELECT STATEMENT   |           |
| 1  | TABLE ACCESS FULL | STUDENT   |
-----
```

3. Analyze the query plan for selecting only student names.

```
EXPLAIN PLAN FOR SELECT Name FROM Student;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
SQL> EXPLAIN PLAN FOR
      2  SELECT Name FROM Student;
```

Explained.

```
SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

PLAN_TABLE_OUTPUT

Id	Operation	Name	

0	SELECT STATEMENT		
1	TABLE ACCESS FULL	STUDENT	

MODEL VIVA QUESTIONS

1. What is a query plan?
2. What is the purpose of EXPLAIN PLAN?
3. What is table scan?
4. How does query optimization improve performance?
5. What is DBMS_XPLAN?

RESULT

Thus the query plan was analyzed successfully using EXPLAIN PLAN.

EXP NO:21**FAMILIARIZATION OF NOSQL DATABASES AND CRUD OPERATIONS****AIM**

To study NoSQL databases and perform CRUD operations.

LEARNING OUTCOMES

- Understand the concept of NoSQL databases.
- Perform Create, Read, Update, and Delete operations.
- Learn basic commands in NoSQL database systems.

DESCRIPTION

A NoSQL database stores data in non-relational formats such as:

- Document databases
- Key-value stores
- Column-family stores
- Graph databases

Example: MongoDB

CRUD operations include:

- Create – Insert data
- Read – Retrieve data
- Update – Modify data
- Delete – Remove data

ALGORITHM

4. Start the NoSQL database server.
5. Create a database and collection.
6. Insert documents into the collection.
7. Retrieve documents.
8. Update a document.
9. Delete a document.

10. Stop.

PROGRAM

use studentDB

db.student.insert({id:1,name:"Anil",marks:80})

db.student.insert({id:2,name:"Ravi",marks:70})

db.student.find()

db.student.update({id:1},{set:{marks:85}})

db.student.remove({id:2})

OUTPUT

```
> use studentDB
switched to db studentDB

> db.student.insert({id:1,name:"Anil",marks:80})
WriteResult({ "nInserted" : 1 })

> db.student.insert({id:2,name:"Ravi",marks:70})
WriteResult({ "nInserted" : 1 })

> db.student.find()
{ "id" : 1, "name" : "Anil", "marks" : 80 }
{ "id" : 2, "name" : "Ravi", "marks" : 70 }

> db.student.update({id:1},{set:{marks:85}})
WriteResult({ "nModified" : 1 })

> db.student.remove({id:2})
WriteResult({ "nRemoved" : 1 })
```

MODEL VIVA QUESTIONS

1. What is NoSQL?
2. What are the types of NoSQL databases?
3. What is CRUD?
4. What is a document database?

5. Give an example of a NoSQL database.

RESULT

Thus the CRUD operations in NoSQL databases were successfully performed.

EXP NO:22**DATABASE APPLICATION USING FRONT END TOOL****AIM**

To design a database application using a front end tool with five or more tables.

LEARNING OUTCOMES

- Design a database application.
- Integrate front end and database.
- Manage data using multiple related tables.

DESCRIPTION

A database application consists of:

- Front End – User interface for input/output.
- Back End – Database storing data.

Example front-end tools include:

- Java
- Visual Basic
- PHP

Example application: Student Management System

Tables used:

1. Student
2. Course
3. Department
4. Faculty
5. Marks

SQL QUERY QUESTIONS

1. Create the Student table.
2. Insert values into the Student table.

3. Create a Department table.
4. Display all students.
5. Display student name and department using join.

SQL QUERY

1. Create the Student table.

```
CREATE TABLE Student (ID NUMBER PRIMARY KEY, Name VARCHAR2(20), DeptID  
NUMBER);
```

```
SQL> CREATE TABLE Student  
2 (ID NUMBER PRIMARY KEY,  
3 Name VARCHAR2(20),  
4 DeptID NUMBER);
```

```
Table created.
```

2. Insert values into the Student table.

```
INSERT INTO Student VALUES(1,'Anil',101);
```

```
INSERT INTO Student VALUES(2,'Ravi',102);
```

```
SQL> INSERT INTO Student VALUES(1,'Anil',101);  
  
1 row created.  
  
SQL> INSERT INTO Student VALUES(2,'Ravi',102);  
  
1 row created.
```

3. Create a Department table.

```
CREATE TABLE Department ( DeptID NUMBER PRIMARY KEY, DeptName  
VARCHAR2(20) );
```

```
SQL> CREATE TABLE Department  
2 (DeptID NUMBER PRIMARY KEY,  
3 DeptName VARCHAR2(20));
```

```
Table created.
```

4. Display all students.

```
SELECT * FROM Student;
```

```
SQL> SELECT * FROM Student;
```

ID	NAME	DEPTID
1	Anil	101
2	Ravi	102

5. Display student name and department using join.

```
SELECT Name,DeptName FROM Student s JOIN Department d ON s.DeptID=d.DeptID;
```

```
SQL> SELECT Name,DeptName  
2 FROM Student s  
3 JOIN Department d  
4 ON s.DeptID=d.DeptID;
```

NAME	DEPTNAME
Anil	Computer
Ravi	Electronics

MODEL VIVA QUESTIONS

1. What is a database application?
2. What is front end and back end?
3. What are primary keys?
4. What is database design?
5. Why are multiple tables used in applications?

RESULT

Thus the database application with multiple tables was successfully designed.